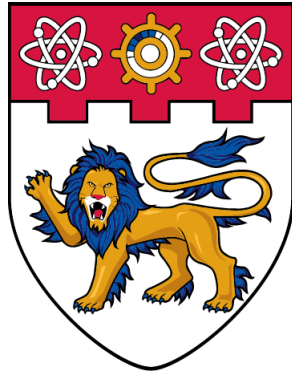


NANYANG TECHNOLOGICAL UNIVERSITY
COLLEGE OF COMPUTING AND DATA SCIENCE



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

**Full Stack Development and
Deployment of Web-based Speech
Recognition System**
By
NGUYEN PHUONG LINH

Project Supervisor: Prof Chng Eng Siong

Singapore
Academic Year 2025/2026

Abstract

This project focuses on the design and implementation of the **Gateway Dashboard**, a comprehensive management interface for the Speech Gateway API, alongside critical backend enhancements to support secure authentication and monetization.

On the **frontend**, a responsive Single Page Application (SPA) was developed using **Vue.js** and **Vuetify**, featuring a modular architecture that ensures scalability and maintainability. Key implemented features include a secure authentication system supporting **Email/Password**, **Google OAuth 2.0**, and **Apple Sign-In**, enhancing user accessibility and security.

The **backend**, built with **NestJS**, was significantly upgraded to include a **Token Management Module** ensuring session security through token blacklisting and automatic expiration handling using Redis and MongoDB TTL indexes.

Furthermore, a **Subscription and Payment Module** was integrated using **Stripe**, enabling automated recurring billing, plan management, and real-time subscription status tracking. Storage solutions utilizing **MongoDB** for persistent data and **Redis** for caching were implemented to optimize performance.

The result is a **production-ready dashboard** that facilitates seamless user onboarding, secure identity management, and a flexible subscription model, thereby establishing a solid foundation for the commercialization of the Speech Gateway services.

Acknowledgement

I would like to express my deepest gratitude to my supervisor, **Chng Eng Siong**, for his invaluable guidance, continuous encouragement, and expert advice throughout the course of this Final Year Project. His insights were instrumental in shaping the direction and reliability of this system.

I am also deeply thankful to my mentors **Kyaw Zin Tun**, **Vu Thi Ly**, and **Surana Tanmay** for their dedicated support and technical mentorship. Their practical knowledge and hands-on guidance greatly contributed to the success of this project.

I extend my sincere thanks to the **Hardware and Embedded Systems Lab** team for providing the necessary resources and technical environment to develop this project. The opportunity to work with real-world technologies like NestJS, Vue.js, and cloud infrastructure has been an immensely rewarding learning experience.

I am also grateful to my friends and family for their unwavering support and patience during the intense periods of development and debugging. Their belief in my abilities kept me motivated to overcome the technical challenges encountered along the way.

Finally, I would like to thank **Nanyang Technological University** for providing the academic foundation that made this work possible.

Contents

Abstract	1
Acknowledgement	2
List of Figures	5
List of Tables	6
1 Introduction	7
1.1 Background	7
1.2 Problem Statement	7
1.3 Project Objectives	7
1.4 Project Scope	8
1.5 Proposed System Overview	8
1.5.1 Single Sign-On (SSO) Authentication	8
1.5.2 Stripe Payment Integration	9
1.5.3 Frontend Dashboard	11
2 Literature Review	13
2.1 Overview	13
2.2 Authentication: Single Sign-On Provider Selection	13
2.2.1 Understanding SSO and Its Role	13
2.2.2 The SSO Market Landscape	13
2.2.3 Selection Rationale: Why Google and Apple	14
2.3 Payment Processing: Selecting the Right Platform	15
2.3.1 The Payment Processing Challenge	15
2.3.2 Comparative Feature Analysis	15
2.3.3 Why Stripe: Strategic Decision	16
2.4 Summary of Technology Decisions	16
3 System Architecture	18
3.1 Overview	18
3.1.1 Three-Tier Architecture	18
3.1.2 Backend Module Organisation	19
3.1.3 Communication Flow	20
3.2 Technology Stack	20
3.2.1 Frontend Design	21
3.2.2 Backend Architecture	21
3.2.3 Data Storage and Caching	22
3.2.4 External Integrations	23

3.3	Database Design	23
3.3.1	Core Data Model	23
3.3.2	Entity Relationships	23
3.3.3	Design Decisions	24
4	Implementation	25
4.1	Backend Implementation	25
4.1.1	Authentication and SSO Implementation	25
4.1.2	Payment & Subscription Management	33
4.2	Frontend Implementation	38
4.2.1	Overview	38
4.2.2	Authentication Frontend	40
4.2.3	Routing and Navigation	41
4.2.4	Subscription Management Interface	46
5	Conclusion and Future Work	49
5.1	Project Summary	49
5.2	Conclusions	49
5.3	Future Work	50
5.4	Final Remarks	50
A		55

List of Figures

1.1	Sign-In Interface with SSO Options	9
1.2	Registration Interface with SSO Integration	9
1.3	Stripe Product Catalogue Configuration	10
1.4	Stripe Checkout Interface for Basic Plan	10
1.5	Subscription Management Interface	11
1.6	Successful Subscription Confirmation	11
1.7	User Management Dashboard	12
1.8	Admin Subscription Management Interface	12
1.9	Plan Upgrade Interface	12
3.1	Three-Tier System Architecture	19
3.2	Backend Module Organisation	19
3.3	Plan Module Caching Architecture	22
3.4	Entity Relationship Diagram	24
4.1	Authentication System Architecture Overview	26
4.2	Google SSO Authentication Flow	27
4.3	Apple Sign-In Authentication Flow	28
4.4	Token Generation During Login	29
4.5	Automatic Token Refresh Mechanism	30
4.6	Logout and Token Revocation Flow	31
4.7	SSO Journey — Existing User	32
4.8	SSO Journey — New User (First-Time Sign-Up)	33
4.9	Payment System Architecture	34
4.10	Stripe Checkout Flow	35
4.11	Subscription Lifecycle State Diagram	36
4.12	Plan Versioning and Subscriber Impact	37
4.13	Stripe Webhook Processing Flow	37
4.14	Frontend Application Architecture	39
4.15	Navigation Guard Decision Flow	43
4.16	Automatic Token Refresh Mechanism	45

List of Tables

2.1	Major SSO Providers and Their Positioning	14
2.2	Stripe and PayPal: Feature and Design Comparison	15
2.3	Gateway Dashboard Technology Decisions and Justification	16
3.1	Technology Stack Summary	21
4.1	Frontend Technology Stack	39
4.2	Route Definitions	42

Chapter 1

Introduction

1.1 Background

In the rapidly evolving landscape of speech processing technologies, the **Speech Gateway API** serves as a critical infrastructure for delivering advanced speech services to various applications. The Speech Gateway is a backend service that provides Speech-to-Text (STT) capabilities to end users through a secured API. It acts as a middle layer between client applications and an underlying Speech Engine, centralising authentication, access control, token issuance, and subscription management.

As the demand for speech processing services grows, the need for a robust, centralised management system becomes paramount. Traditionally, interacting with such APIs required direct integration and manual management, which can be inefficient for both end-users and administrators. The **Gateway Dashboard** project was initiated to bridge this gap, providing a user-friendly graphical interface that empowers users to manage their speech processing tasks while giving administrators the tools needed to oversee system operations and monetise services effectively.

1.2 Problem Statement

Prior to this project, the Speech Gateway ecosystem faced two main challenges. First, the existing system had **limited access control**, supporting only basic credentials without modern Single Sign-On (SSO) capabilities. Users had to create separate accounts and manage passwords manually, leading to credential fatigue and potential security risks from weak or reused passwords. Second, there was a complete **absence of monetisation infrastructure**—no automated system existed to handle subscriptions, billing, or tiered access plans, which hindered the commercial viability of the services.

1.3 Project Objectives

The primary objective of this project is to design and implement a comprehensive Gateway Dashboard and enhance the backend infrastructure to support commercial-grade features. The specific objectives are:

- **SSO Integration:** Implement secure Single Sign-On with Google and Apple to streamline user authentication, reduce credential fatigue, and centralise identity management across services.

- **Payment Portal:** Design a subscription and payment module integrating Stripe to handle recurring billing, automated invoicing, and subscription lifecycle management.
- **Frontend Development:** Develop a web-based dashboard using Vue.js and Vuetify to provide an intuitive interface for users to access speech services and manage their accounts.

1.4 Project Scope

The project encompasses three major components that together form a complete platform for managing the Speech Gateway services.

The **SSO Authentication** component enables users to authenticate using their existing Google or Apple accounts, in addition to traditional email and password credentials. This includes secure token management with automatic refresh mechanisms, token blacklisting via Redis for immediate revocation, and session security measures to prevent unauthorised access.

The **Payment Portal with Stripe Integration** component handles monetisation through the Stripe payment platform, enabling tiered subscription plans with usage-based quotas. It includes hosted checkout sessions, webhook-based event processing for payment lifecycle events, versioned plan management with an admin interface, and real-time synchronisation between Stripe and the internal database.

The **Frontend Dashboard** component provides a complete Single Page Application (SPA) built with Vue.js and Vuetify. It features authentication pages with SSO integration, protected routing with role-based access control, subscription management with usage monitoring, and an admin interface for managing plans and users.

1.5 Proposed System Overview

This section presents the three main components of the proposed system, illustrating how each addresses the challenges identified in the problem statement.

1.5.1 Single Sign-On (SSO) Authentication

Before: Users needed to create separate credentials for the Speech Gateway, leading to credential fatigue and security risks from weak or reused passwords.

After: Users can now sign in with their existing Google or Apple accounts in a streamlined flow. The user clicks on an SSO button, the system redirects to the identity provider for authentication, and upon success, the user is automatically logged in and redirected to the dashboard. For first-time SSO users, a brief account completion step collects any additional required information before granting full access.

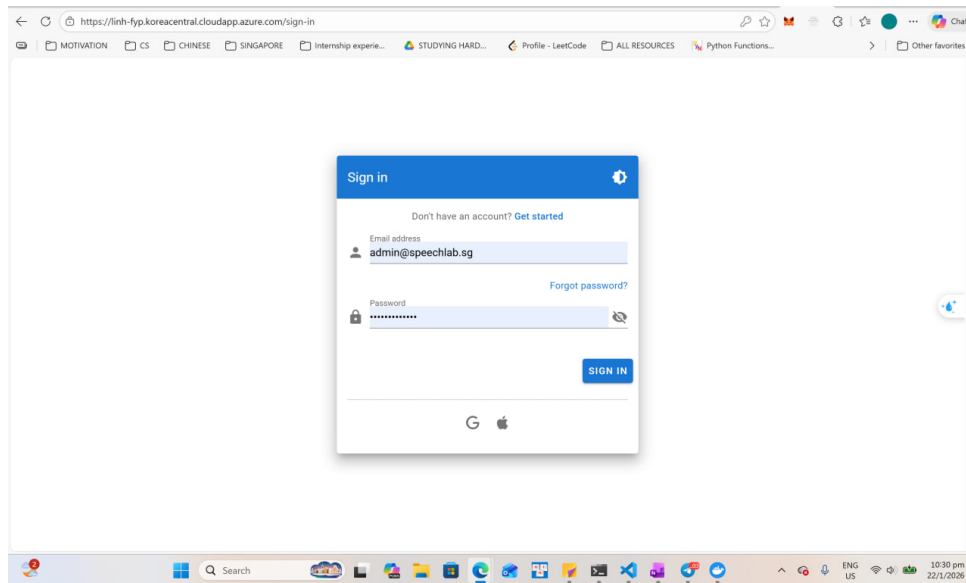


Figure 1.1: Sign-In Interface with SSO Options

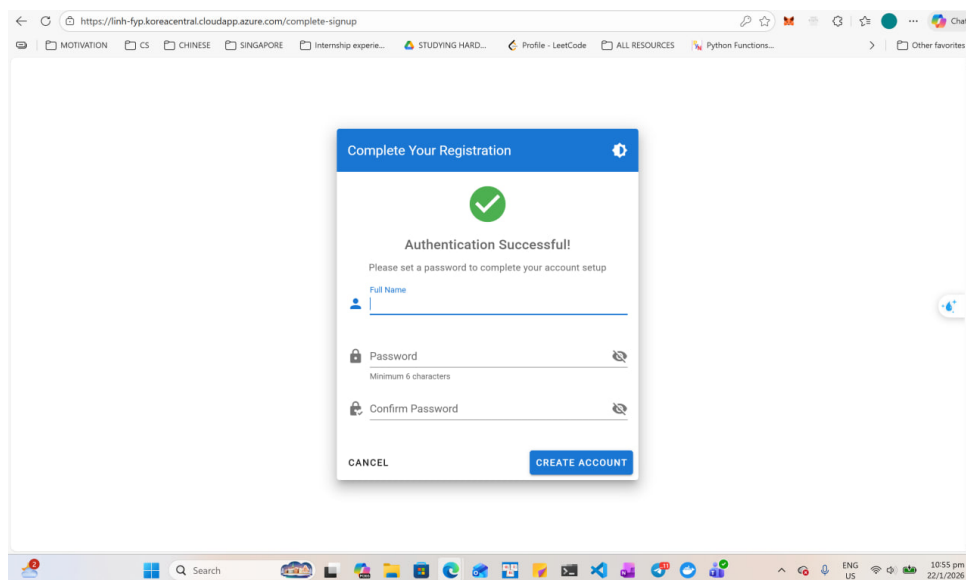


Figure 1.2: Registration Interface with SSO Integration

1.5.2 Stripe Payment Integration

Before: No automated billing existed, requiring manual invoice generation and payment verification before granting access to speech services.

After: The system now supports a fully automated payment flow. Administrators configure subscription plans in the Stripe product catalogue. When a user selects a plan and subscribes, the system creates a secure checkout session through Stripe's hosted page. Upon successful payment, Stripe notifies the system via webhooks, and the user's subscription status is automatically updated with the appropriate service quotas.

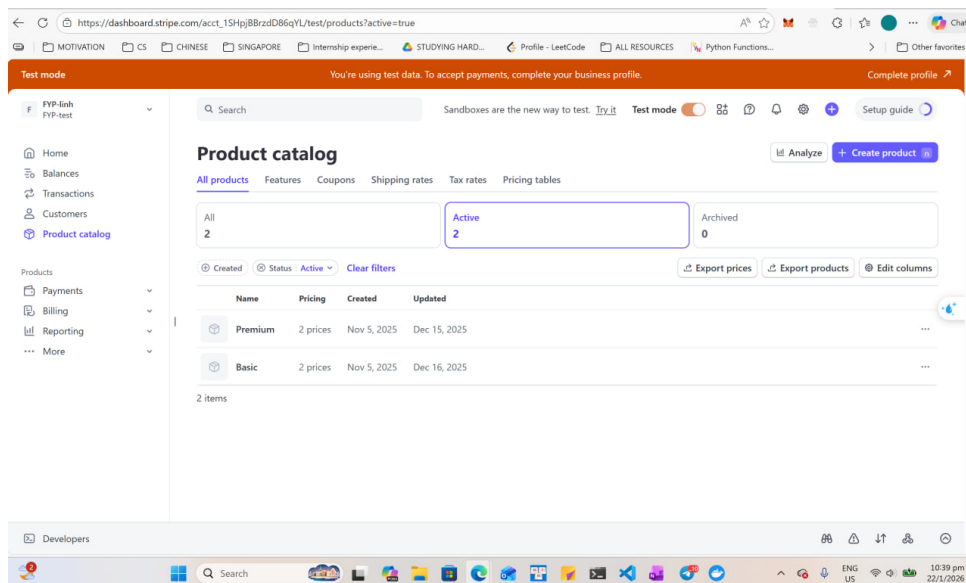


Figure 1.3: Stripe Product Catalogue Configuration

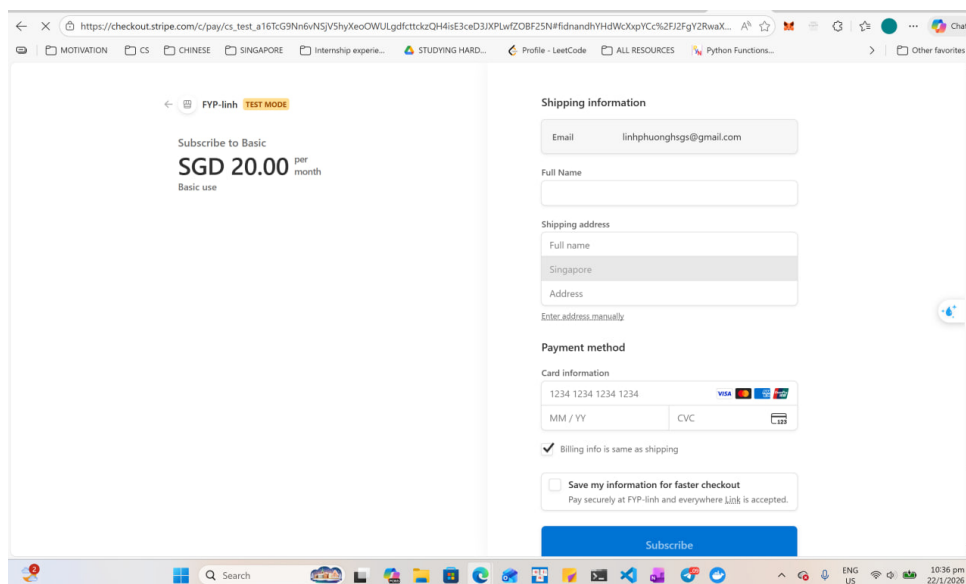


Figure 1.4: Stripe Checkout Interface for Basic Plan

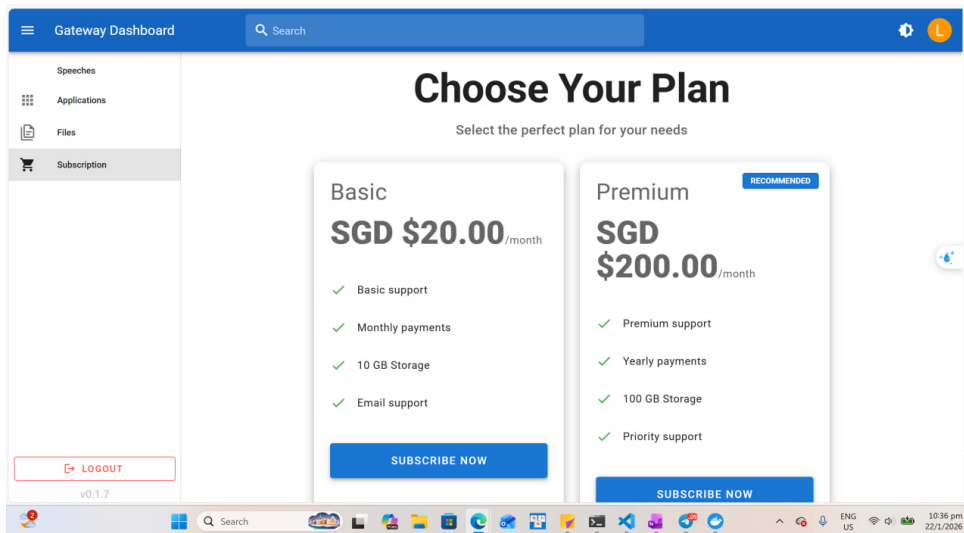


Figure 1.5: Subscription Management Interface

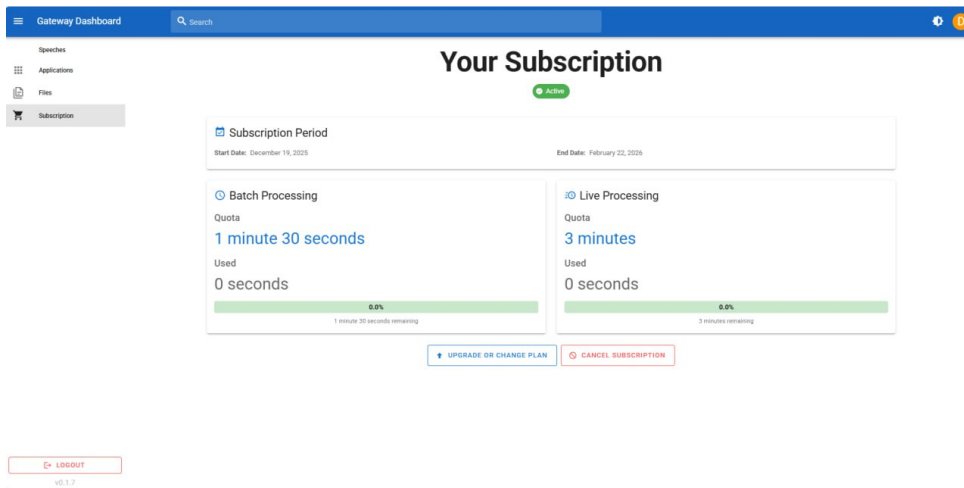


Figure 1.6: Successful Subscription Confirmation

1.5.3 Frontend Dashboard

Before: No visual interface existed, requiring direct API calls for all operations and making it difficult to monitor subscriptions or manage users.

After: The dashboard provides a complete user experience. After authenticating via SSO or traditional login, users can view their subscription status and usage quotas, manage their profile, and upgrade or change plans. Administrators have access to additional tools for managing users, configuring subscription plans, and monitoring the overall system.

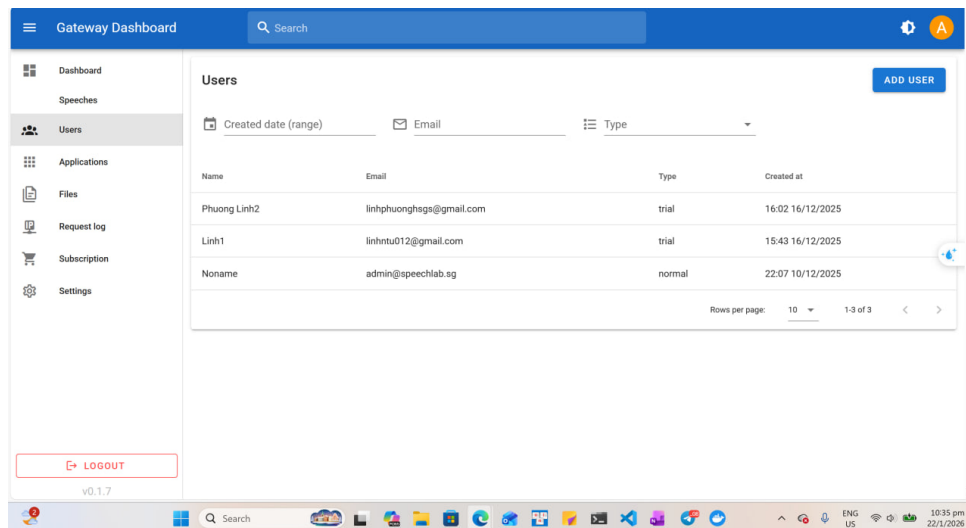


Figure 1.7: User Management Dashboard

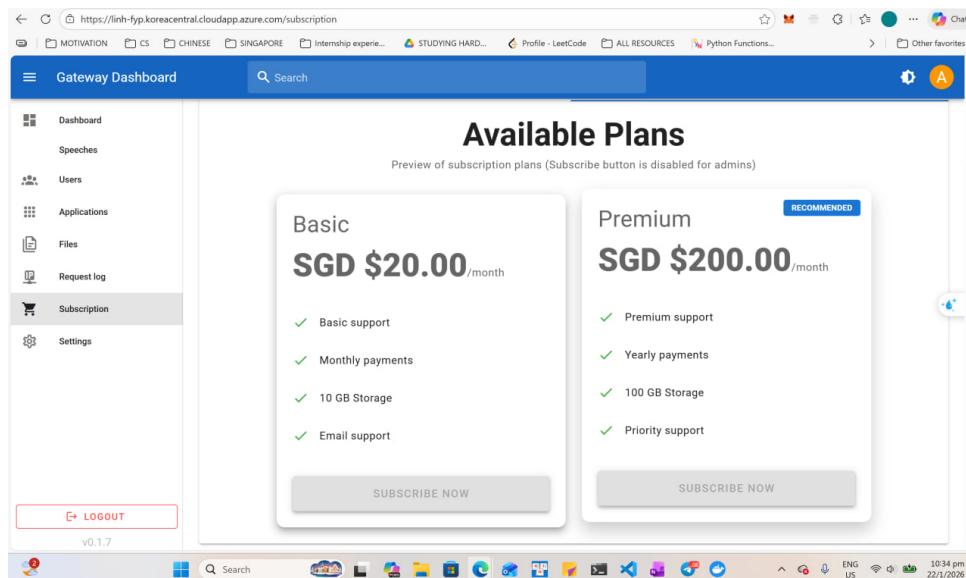


Figure 1.8: Admin Subscription Management Interface

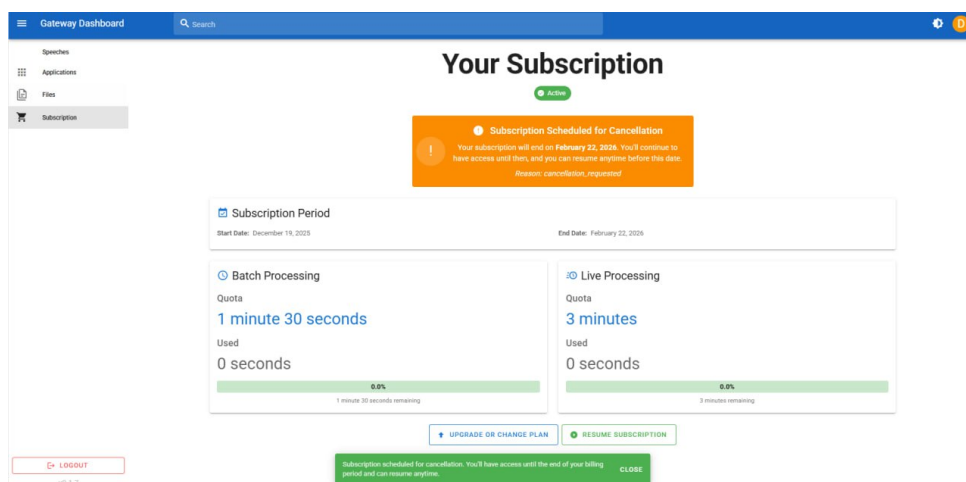


Figure 1.9: Plan Upgrade Interface

Chapter 2

Literature Review

2.1 Overview

The Gateway Dashboard project integrates two critical external systems: user authentication through Single Sign-On (SSO) providers, and payment processing for API usage billing. This chapter examines the technology decisions for these integrations, focusing on how market landscape, user base alignment, and API maturity informed the selection of Google and Apple for authentication, and Stripe for payment processing.

These choices reflect the broader principle underlying the Gateway Dashboard: prioritizing developer experience and operational reliability in the API management domain, while ensuring security and scalability for real-world deployment.

2.2 Authentication: Single Sign-On Provider Selection

2.2.1 Understanding SSO and Its Role

Single Sign-On (SSO) enables users to authenticate once and gain access to multiple applications or services using a single set of credentials. Rather than requiring users to create and maintain separate usernames and passwords for each system, SSO delegates authentication to a trusted identity provider—such as Google or Apple—that the user already trusts and uses regularly.

For the Gateway Dashboard, implementing SSO serves two important purposes. First, it reduces friction for new users: developers already have Google or Apple accounts, so onboarding requires only a single click rather than filling out registration forms. Second, it outsources security responsibilities to platforms with significant resources dedicated to authentication infrastructure, including fraud detection, account recovery, and multi-factor authentication.

2.2.2 The SSO Market Landscape

Several identity providers compete for dominance in the SSO space, each with different strengths. Table 2.1 presents the major options and their characteristics:

Table 2.1: Major SSO Providers and Their Positioning

Provider	User Base / Focus	Distinctive Feature
Google	2B+ users (71% Android market share)	Ubiquitous, well-established OAuth 2.0 API
Apple	28% iOS market share	Privacy-first design, email relay feature
Microsoft	Enterprise and productivity users	Azure AD integration for organizations
Facebook	2.9B users	Large social graph
GitHub	Developer community	Native integration for tech professionals

2.2.3 Selection Rationale: Why Google and Apple

The decision to implement Google and Apple SSO emerged from analysis of the target user base and strategic priorities:

1. **Market Coverage and Accessibility:** Over 95% of internet users possess at least one Google or Apple account [50, 49]. This means the vast majority of potential Gateway Dashboard users can authenticate with minimal additional steps. Requiring users to set up Microsoft, Facebook, or GitHub accounts would impose unnecessary barriers.
2. **Developer Alignment:** The Gateway Dashboard serves software developers building API-driven applications. Surveys indicate that developers are more likely to have Google and Apple accounts for personal use (mobile devices, productivity) compared to enterprise-specific solutions. This alignment between user behavior and available authentication methods maximizes conversion during onboarding.
3. **Technical Maturity and Reliability:** Both Google and Apple offer production-grade OAuth 2.0 implementations with 99.9%+ uptime guarantees [20, 2]. Their authentication infrastructure handles billions of login events daily, meaning the systems are battle-tested and unlikely to introduce unexpected downtime affecting the Gateway Dashboard.
4. **Security and Privacy:** Both providers implement industry-standard security practices: two-factor authentication, device trust verification, and anomaly detection. Additionally, Apple’s privacy-focused philosophy—including its email relay feature that shields user email addresses—aligns with growing regulatory expectations around user data protection.
5. **Team Capacity and Scope:** The project divided SSO responsibilities among team members. This choice avoided duplicating authentication provider implementations across the team and allowed parallel, independent work streams.

The omission of Facebook and GitHub reflects pragmatic scoping: while GitHub would appeal to developers, it serves a smaller addressable market. Facebook, despite its large user base, is less aligned with professional developer workflows.

2.3 Payment Processing: Selecting the Right Platform

2.3.1 The Payment Processing Challenge

A payment platform is more than a transaction processor—it is the operational backbone for billing and revenue. Beyond accepting one-time payments, modern SaaS platforms like the Gateway Dashboard must handle recurring subscriptions, metered billing based on usage, failed payment retries, and detailed financial reporting.

Stripe and PayPal represent the two most mature options for this category. Both support similar transaction fees (2.9% + \$0.30 per transaction) and reach sufficient merchant and customer bases. Yet they differ significantly in their feature depth, developer ergonomics, and alignment with subscription-based business models.

2.3.2 Comparative Feature Analysis

Table 2.2 outlines the key dimensions distinguishing Stripe and PayPal:

Table 2.2: Stripe and PayPal: Feature and Design Comparison

Dimension	Stripe	PayPal
Transaction Fee	2.9% + \$0.30 per transaction	2.9% + \$0.30 per transaction
API Design	RESTful API with comprehensive, accessible documentation [51]	Multiple API generations; historically complex migration paths
Checkout Experience	Embedded checkout (user stays on merchant site)	Redirect-based checkout (user leaves merchant site)
Checkout Conversion	Maintains user context, higher completion rates	Redirect flow typically reduces completion by 10-15% [6]
Subscription Billing	Native, production-ready: tiered plans, prorated changes, automated renewal [52]	Limited flexibility; less mature subscription support
Metered Billing	Supported natively	Not supported
Webhook System	Automatic retry with exponential backoff; cryptographic signature verification [48]	Basic webhook support
Integration Speed	Approximately 40% faster typical implementation [24]	Slower; documentation fragmentation increases learning curve
Developer Experience	Strong focus; SDKs in multiple languages	Limited developer-first tooling

2.3.3 Why Stripe: Strategic Decision

Stripe was selected for the Gateway Dashboard’s payment infrastructure for reasons aligned with the project’s architecture and business model:

- **Subscription-Native Design:** The Gateway Dashboard monetizes through usage-based subscriptions (API request quotas, tiered pricing). Stripe’s billing system natively supports this model with prorated upgrades, quota management, and automated renewal—features that PayPal does not offer cleanly. Implementing these on PayPal would require custom workarounds.
- **Developer Experience and Speed:** Stripe’s RESTful API and well-organized documentation reduce time-to-integration. The provided Node.js SDK aligns with the project’s backend stack, minimizing custom code. This efficiency matters in an academic timeline with fixed deadlines.
- **Embedded Checkout Flow:** Stripe’s embedded checkout keeps users within the Gateway Dashboard throughout the payment process. Redirecting to PayPal’s payment page, by contrast, risks checkout abandonment—studies show 10-15% conversion loss when users leave a merchant site [6]. For an API management SaaS, maintaining contextual awareness during payment increases user confidence.
- **Webhook Reliability and Consistency:** Payment confirmations must reliably reach the Gateway Dashboard’s database to activate API access. Stripe’s webhook system includes automatic retry logic with exponential backoff and cryptographic verification. This design prevents payment-database inconsistencies that could lock out paying customers or grant free access to non-payers.
- **Audience Alignment:** The Gateway Dashboard’s users are software developers. Developers prioritize technical functionality, API quality, and developer-friendly tooling—Stripe’s core strength—over consumer familiarity with brand names like PayPal.

PayPal remains a valid choice for other use cases (especially consumer-facing, one-time transactions), but subscription-centric platforms like the Gateway Dashboard benefit significantly from Stripe’s design philosophy.

2.4 Summary of Technology Decisions

Table 2.3 consolidates the key findings of this literature review:

Table 2.3: Gateway Dashboard Technology Decisions and Justification

Component	Selection	Primary Justification
SSO Providers	Google + Apple	95%+ user coverage; strong developer alignment; enterprise-grade reliability
Payment Platform	Stripe	Subscription-native design; superior developer UX; embedded checkout; reliable webhooks

These decisions reflect a consistent design philosophy: choosing technologies that balance user accessibility (broad authentication options, smooth payment flow) with technical maturity and developer experience. This approach ensures the Gateway Dashboard is both approachable for new users and maintainable for the development team within realistic time constraints.

Chapter 3

System Architecture

This chapter presents the Gateway Dashboard system architecture at a high level, focusing on how the key components interact and the design decisions that enable the platform to scale and evolve. Rather than diving into implementation details, we examine the technology choices, communication patterns, and data management approach that professors and stakeholders need to understand the system's overall design.

3.1 Overview

The Gateway Dashboard system follows a modern three-tier architecture designed for clarity, scalability, and maintainability. The system separates concerns across three distinct layers: a user-facing frontend built with Vue.js, a business logic layer implemented in NestJS, and a persistent data layer comprising MongoDB and Redis. This separation allows each layer to evolve independently while maintaining clear, well-defined communication boundaries.

3.1.1 Three-Tier Architecture

Figure 3.1 illustrates the three-tier architecture and the external services integrated into the system.

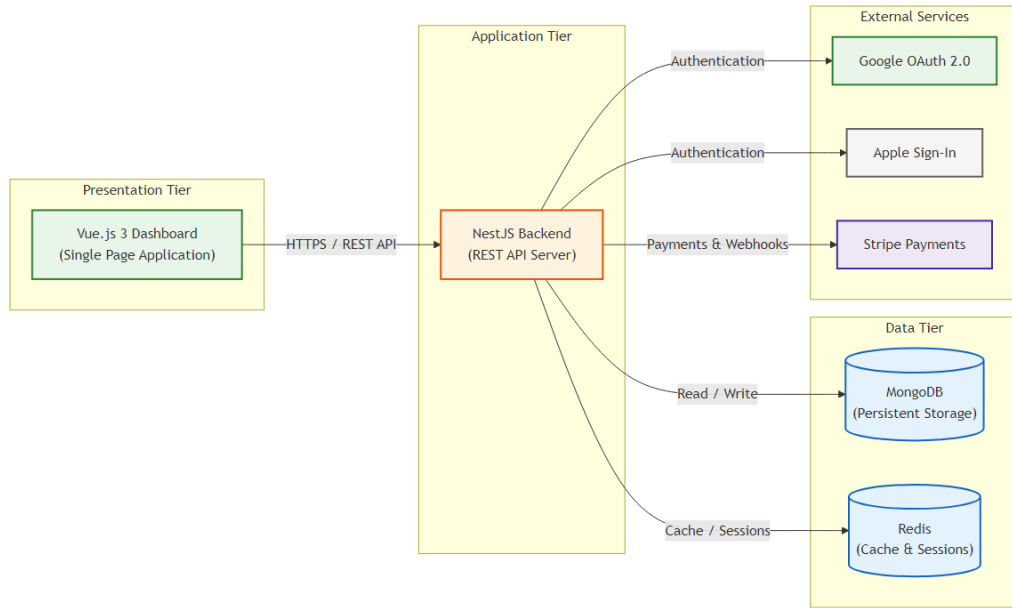


Figure 3.1: Three-Tier System Architecture

The **Presentation Tier** is a Single Page Application built with Vue.js 3, running entirely in the user's browser. It communicates with the backend through standard HTTPS REST API calls. The **Application Tier** is a NestJS server that processes all business logic, including authentication, payment handling, and subscription management. The **Data Tier** uses MongoDB for persistent storage of user accounts, subscriptions, and plan definitions, alongside Redis for caching frequently accessed data and managing user sessions.

Three external services are integrated: **Google OAuth 2.0** and **Apple Sign-In** provide SSO authentication, while **Stripe** handles all payment processing and sends real-time event notifications back to the backend via webhooks.

3.1.2 Backend Module Organisation

Figure 3.2 shows how the backend is organised into distinct modules, each responsible for a specific area of functionality.

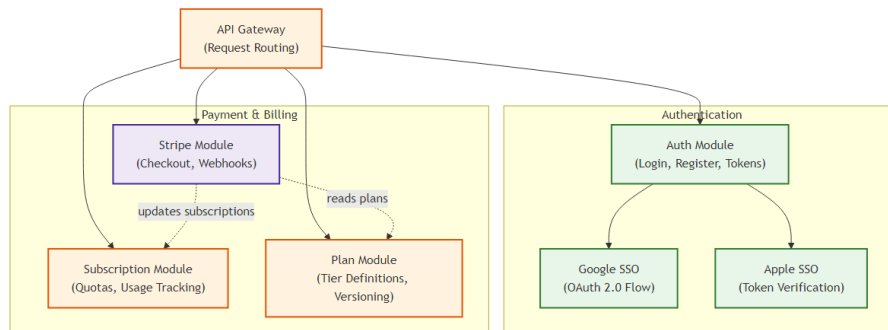


Figure 3.2: Backend Module Organisation

The API Gateway routes incoming requests to the appropriate module. The **Authentication** group handles user login, registration, and token management, with dedicated sub-modules for Google SSO (using the OAuth 2.0 redirect flow) and Apple SSO (using

token verification). The **Payment & Billing** group consists of the Stripe Module (which manages checkout sessions and processes payment webhooks), the Subscription Module (which tracks each user’s subscription status and usage quotas), and the Plan Module (which stores versioned plan definitions and caches them for performance). The dotted lines indicate cross-module communication: the Stripe Module reads plan definitions when creating checkout sessions and updates subscription records when payment events occur.

3.1.3 Communication Flow

User interactions flow naturally through the system: when a user interacts with the frontend, the browser sends HTTP requests to the backend API, where incoming requests are validated and authorised. The backend then executes the required business logic, which may involve retrieving or storing data, checking cached values, or communicating with external payment and authentication services. Once processing is complete, the backend responds with the result, allowing the frontend to update the user interface accordingly.

3.2 Technology Stack

The Gateway Dashboard leverages modern, industry-standard technologies chosen for their reliability, maturity, and ability to support scalable applications. The following table summarizes the key technologies across all system layers:

Layer	Technology	Purpose
Frontend	Vue.js 3	Progressive JavaScript framework with reactive UI components and excellent developer experience
Frontend	Vuetify 3	Material Design component library enabling rapid development of professional interfaces
Frontend	Vue Router 4	Client-side routing for seamless single-page application navigation
Frontend	Axios	HTTP client for communicating with the backend API
Backend	NestJS 10	Modular Node.js framework providing structure and built-in dependency injection
Backend	TypeScript 5	Typed superset of JavaScript ensuring code reliability and maintainability
Backend	Passport	Flexible authentication middleware supporting multiple OAuth providers
Backend	JWT	Stateless authentication tokens enabling secure, scalable session management
Data Storage	MongoDB 6	Document database with flexible schema, ideal for evolving application requirements
Caching	Redis 7	In-memory data store providing fast access to frequently requested data
External	Stripe	Payment processing platform handling subscriptions, billing, and transactions
External	Google OAuth 2.0	Identity provider enabling social login via Google accounts
External	Apple Sign-In	Authentication service supporting Apple device users

Table 3.1: Technology Stack Summary

3.2.1 Frontend Design

The frontend is built with Vue.js 3, which provides a modern reactive framework with excellent TypeScript support. Vue Router handles navigation between different pages, while Axios manages all communication with the backend. Vuetify components enable rapid development while maintaining a consistent, professional Material Design appearance across the application.

3.2.2 Backend Architecture

The backend is implemented with NestJS, a progressive Node.js framework that enforces modular, scalable architecture patterns. TypeScript ensures type safety throughout the codebase, reducing runtime errors and improving code maintainability. Authentication is managed through Passport middleware combined with JWT tokens, allowing stateless, scalable session handling without server-side session storage.

3.2.3 Data Storage and Caching

MongoDB serves as the primary data store, offering flexible document-based storage that adapts as the application evolves. To optimize performance, frequently accessed data—such as service plans and pricing information—is cached in Redis. This caching strategy reduces database load and API latency by checking the fast in-memory cache first, falling back to the database only when the cache is empty. When a cache miss occurs, the retrieved data is stored for future requests, creating a simple yet effective read-through cache pattern.

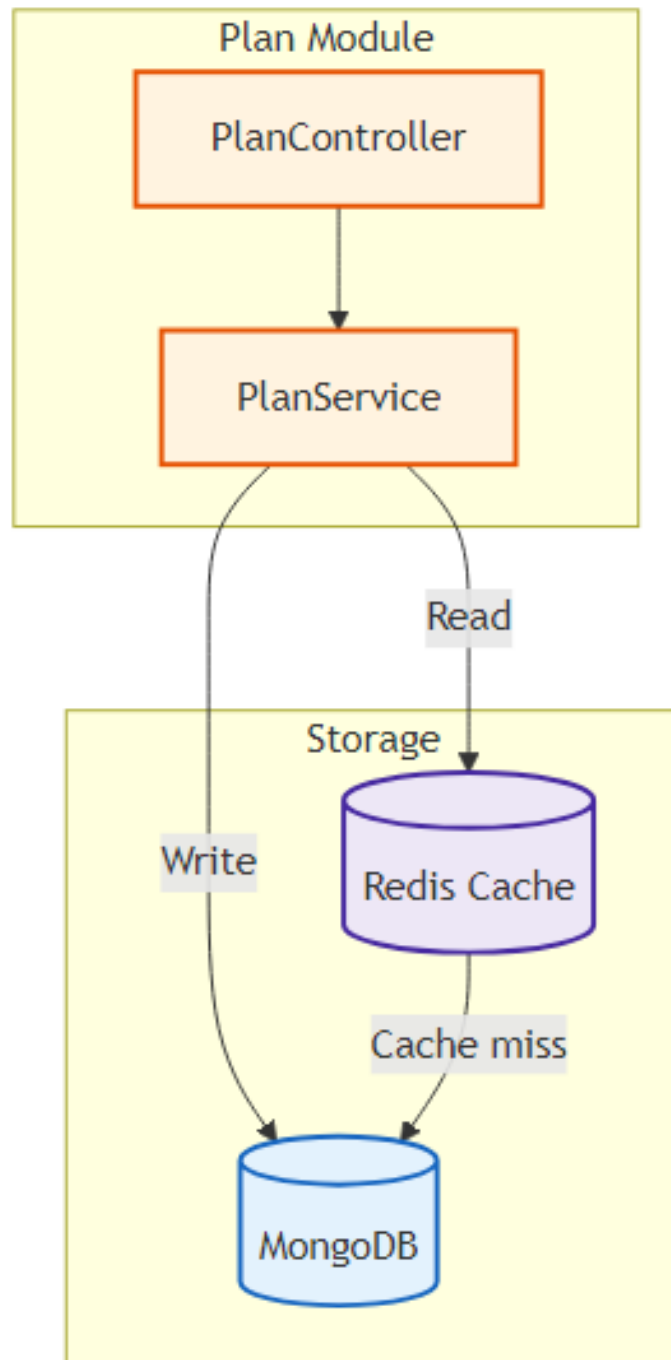


Figure 3.3: Plan Module Caching Architecture

This approach significantly improves response times for frequently requested data while minimizing unnecessary database queries during peak usage.

3.2.4 External Integrations

The system integrates with three key external services. Google OAuth and Apple Sign-In provide secure, passwordless authentication, reducing the application's authentication overhead while leveraging trusted identity providers. Stripe handles all payment processing, including subscription creation, billing cycles, and transaction management, ensuring compliance with PCI standards and industry best practices.

3.3 Database Design

The database schema is designed to support user account management, secure authentication, subscription tracking, and flexible service plan versioning. MongoDB's document-oriented approach provides the flexibility needed for an evolving application while maintaining data consistency and relationships.

3.3.1 Core Data Model

The system manages several interconnected data domains. The **Users** collection is the foundation, storing user account information including email, authentication credentials, user roles, and subscription status. Each user account can have an associated **Subscription**, which tracks their current service plan, usage quotas, and billing integration with external payment systems. The **Plans** collection maintains versioned service offerings, allowing the business to introduce new plans or modify existing ones without disrupting active subscriptions.

To support secure session management, two additional collections manage authentication state. The **UserTokens** collection tracks active authentication tokens for each user session, storing metadata such as token expiration time and client information. The **TokenBlacklist** collection maintains a record of revoked tokens, ensuring that logged-out users or users with invalidated sessions cannot reuse their old authentication credentials. Both collections leverage MongoDB's automatic expiration (TTL) feature to clean up old entries without manual intervention.

3.3.2 Entity Relationships

Figure 3.4 illustrates how these collections relate to one another:

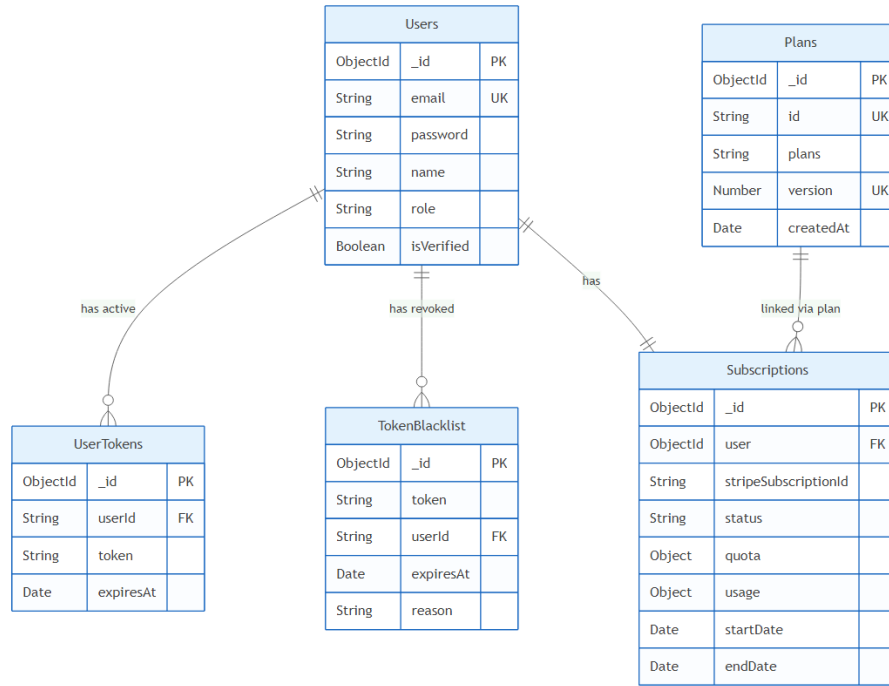


Figure 3.4: Entity Relationship Diagram

The data model reflects common business logic. Each user maintains at most one active subscription, creating a one-to-one relationship. A subscription references an available plan and integrates with Stripe to track billing information. Users may have multiple active authentication tokens (for different devices or sessions), creating a one-to-many relationship. When users log out or sessions are invalidated, their tokens are moved to the blacklist to prevent reuse. The Plans collection exists independently but is logically referenced by subscriptions, enabling the application to manage plan availability and pricing changes without affecting existing customer data.

3.3.3 Design Decisions

The database schema reflects several key design principles. By storing user account information, subscription details, and authentication tokens in separate collections, the schema maintains clean separation of concerns—account data, billing data, and session management can be queried and updated independently. The use of document-based storage allows subscription records to embed service quota and usage information, making it efficient to retrieve complete subscription details in a single database operation. Token expiration handling through TTL indexes eliminates the need for periodic cleanup jobs, reducing operational complexity. This design balances normalization (separating logically distinct concepts) with denormalization (embedding related data to minimize joins), a practical approach for document databases like MongoDB.

Chapter 4

Implementation

This chapter describes the implementation of the Gateway Dashboard system, organised into two main parts. The first part covers the backend services, including the authentication system with Single Sign-On support and the payment and subscription management infrastructure. The second part covers the frontend dashboard application built with Vue.js.

4.1 Backend Implementation

The backend is built with NestJS, a modular Node.js framework that provides a well-structured foundation for building scalable server-side applications. The backend is organised into distinct modules, each responsible for a specific domain of functionality. This section describes the two major backend subsystems developed in this project: the authentication system and the payment and subscription management system.

4.1.1 Authentication and SSO Implementation

This section describes the authentication system implemented in the Gateway Dashboard. The system integrates three main modules that work together to provide secure user authentication, whether through traditional email/password login or modern single sign-on with Google and Apple.

Figure 4.1 provides an overview of how these authentication modules integrate within the system.

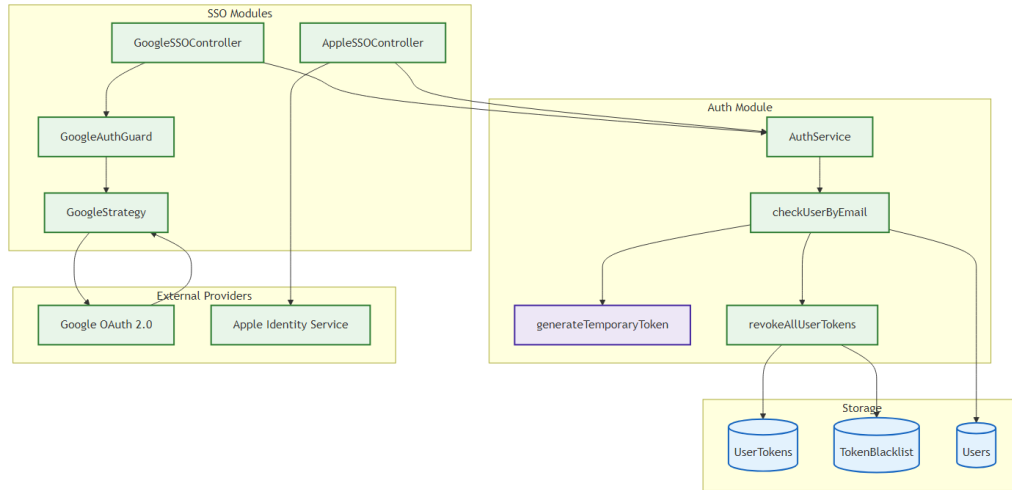


Figure 4.1: Authentication System Architecture Overview

The architecture shows three main components: the core Auth Module handling traditional login, the Google SSO Module using OAuth 2.0 redirects, and the Apple SSO Module using token-based verification. All three modules share a common token management layer that issues and validates user sessions.

4.1.1.1 Google SSO Implementation

Google SSO enables users to authenticate using their existing Google accounts through the OAuth 2.0 standard protocol. This approach eliminates the need for users to create and remember another password while leveraging Google’s robust identity verification infrastructure.

4.1.1.1.1 OAuth 2.0 Flow Figure 4.2 illustrates the simplified Google authentication flow.

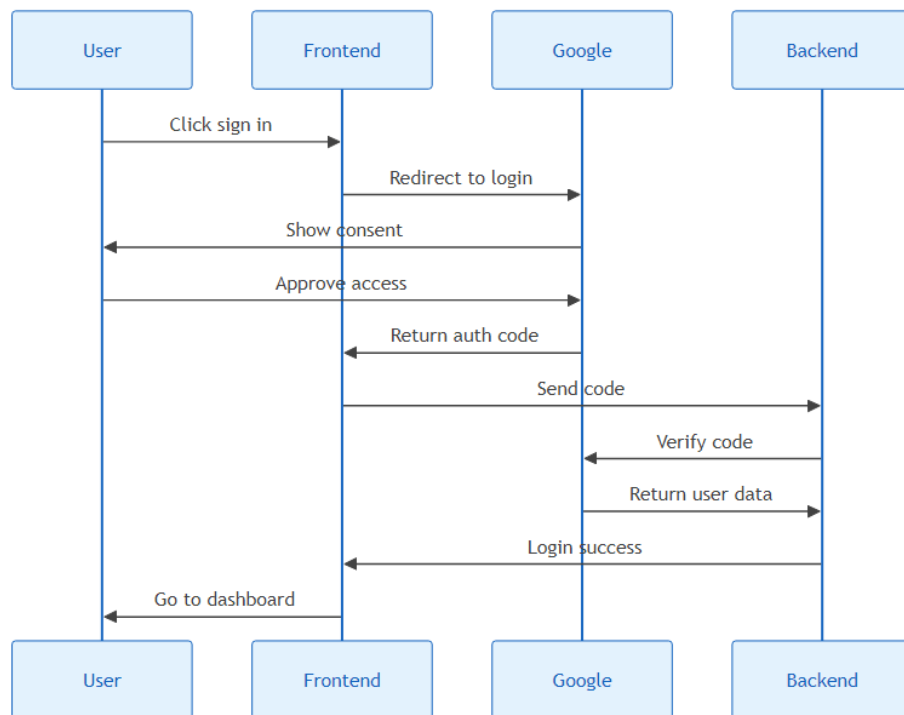


Figure 4.2: Google SSO Authentication Flow

The flow proceeds as follows: when a user clicks “Sign in with Google,” the frontend redirects them to Google’s consent screen. The user reviews the requested permissions (email and profile access) and approves. Google then sends an authorisation code back to the backend, which exchanges it for the user’s profile information. The backend checks whether this Google account is already linked to an existing user—if so, the user is logged in immediately; if not, they are directed to a brief signup completion page.

A key design decision involves preserving user context during the redirect. Before sending the user to Google, the system encodes the user’s intended destination into a state parameter that travels with the request. When Google redirects back, this state is decoded to resume the user’s original action, preventing context loss during the multi-step authentication process.

4.1.1.2 Apple SSO Implementation

Apple Sign-In provides an alternative single sign-on option for users with Apple accounts, particularly important for iOS applications.

4.1.1.2.1 POST Callback Architecture Unlike Google’s redirect-based OAuth flow, Apple Sign-In uses a POST callback mechanism. After the user approves sign-in on their Apple device, Apple sends the authentication response directly to the backend via an HTTP POST request rather than redirecting the user’s browser. This POST-based approach provides cleaner handling of the user’s session and reduces potential issues with browser redirects.

Figure 4.3 illustrates the Apple Sign-In flow.

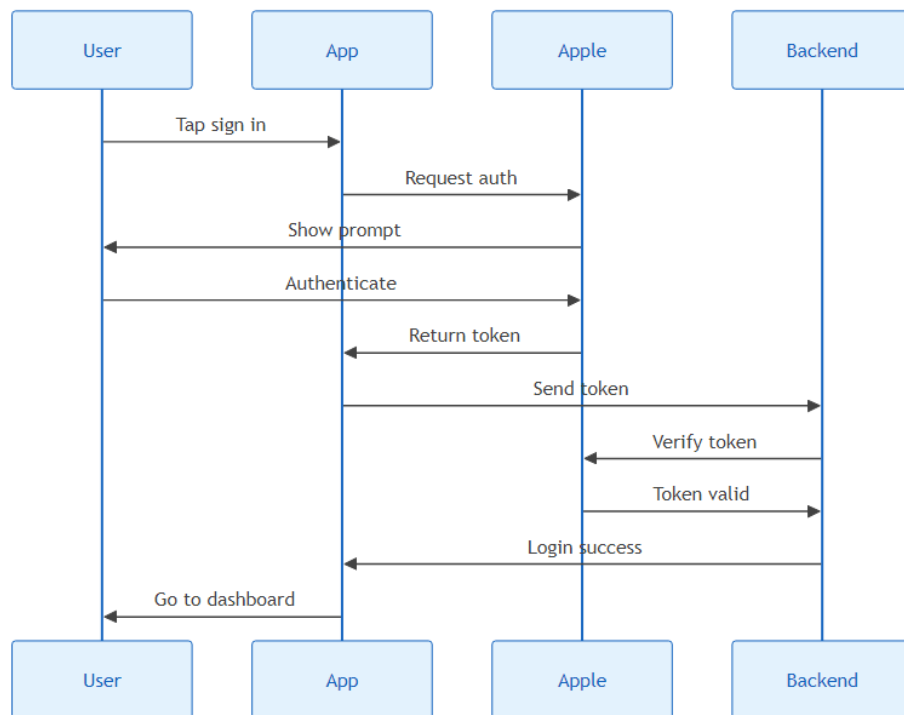


Figure 4.3: Apple Sign-In Authentication Flow

The flow begins when the user taps “Sign in with Apple” on their device. Apple authenticates the user (via Face ID, Touch ID, or passcode) and returns a signed identity token to the application. The app forwards this token to the backend, which verifies its authenticity with Apple’s servers. Similar to the Google flow, the backend then checks whether the user already has an account. Existing users are logged in immediately, while new users are directed to complete their account setup.

The backend implements a two-tier verification strategy for Apple’s identity tokens: it first attempts verification using an optimised library, and if that fails, falls back to manual cryptographic verification. This dual approach ensures robust token validation even if external dependencies become temporarily unavailable.

4.1.1.3 Token Management

The system issues JSON Web Tokens (JWTs) to represent authenticated user sessions. A JWT is a digitally signed, self-contained token that encodes information about the user and has a limited validity period. The token management system handles three key operations: generation, refresh, and revocation.

4.1.1.3.1 Token Generation (Login) Figure 4.4 shows what happens when a user logs in.

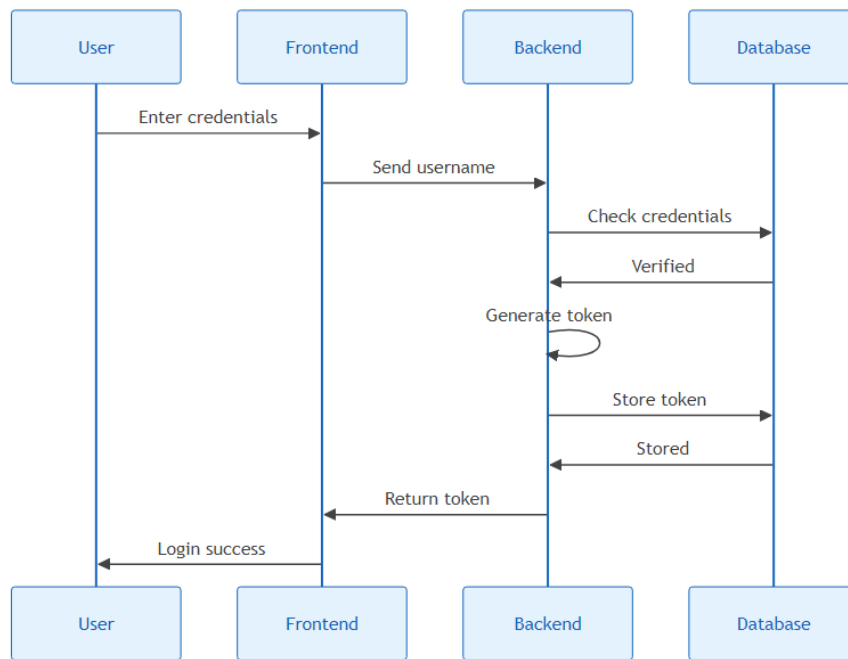


Figure 4.4: Token Generation During Login

When a user submits their credentials, the backend validates them against the database. If valid, any existing tokens for that user are invalidated (enforcing a single-session policy), a new token is generated and stored, and the token is returned to the frontend. This ensures only one active session exists per user at any time.

4.1.1.3.2 Token Refresh Tokens have short lifespans to limit damage if compromised. To avoid forcing users to re-authenticate frequently, the system implements automatic token refresh.

Figure 4.5 illustrates this mechanism.

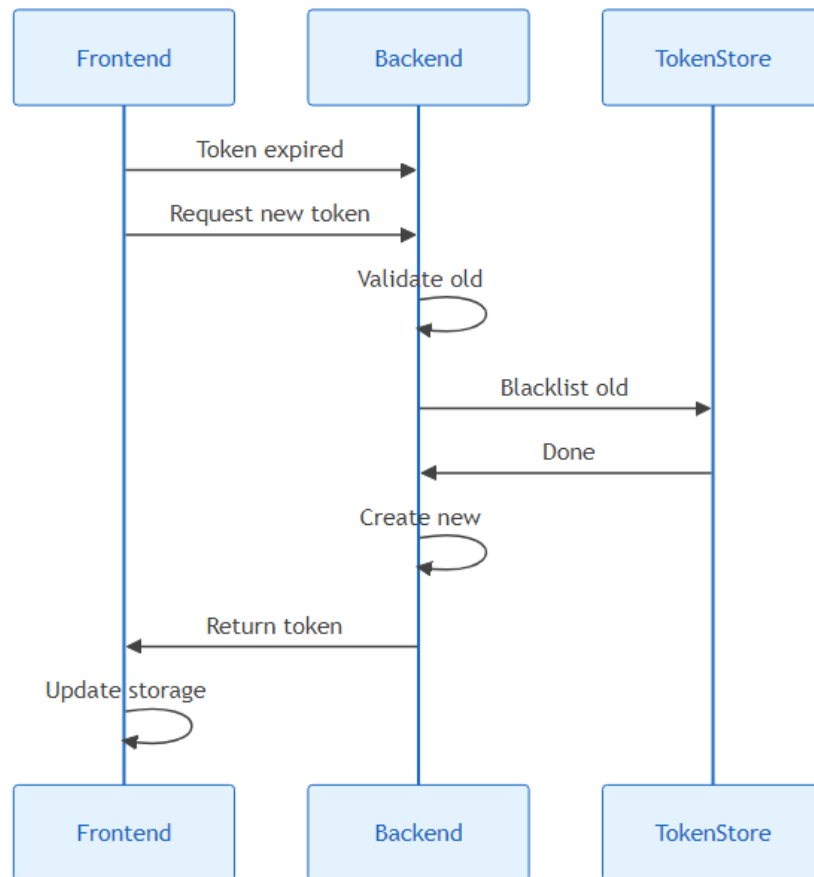


Figure 4.5: Automatic Token Refresh Mechanism

When a token approaches expiration, the frontend sends a refresh request. The backend validates the expiring token, immediately adds it to a blacklist to prevent reuse, generates a fresh token, and returns it. This cycle continues transparently, keeping the user logged in without interruption while maintaining security through short-lived credentials.

4.1.1.3.3 Logout and Token Revocation Figure 4.6 shows the logout process.

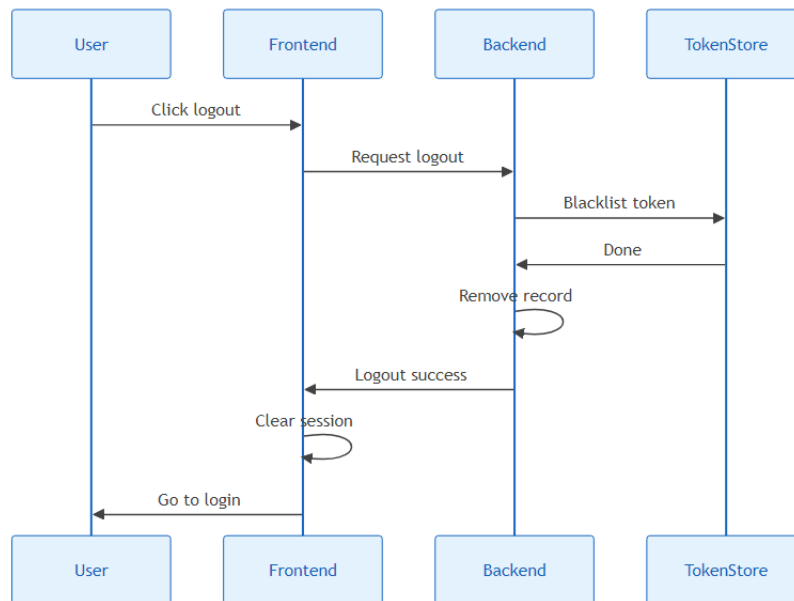


Figure 4.6: Logout and Token Revocation Flow

When a user logs out, the backend adds their current token to a blacklist stored in Redis (enabling instant revocation) and removes the token record from the database. The frontend then clears any locally stored session data and redirects the user to the sign-in page. Any subsequent attempt to use the revoked token is rejected.

4.1.1.4 End-to-End SSO User Journey

The SSO user journey differs depending on whether the user already has an account. The following two diagrams illustrate each path separately.

4.1.1.4.1 Existing User Journey Figure 4.7 shows the flow for users who already have an account in the system.

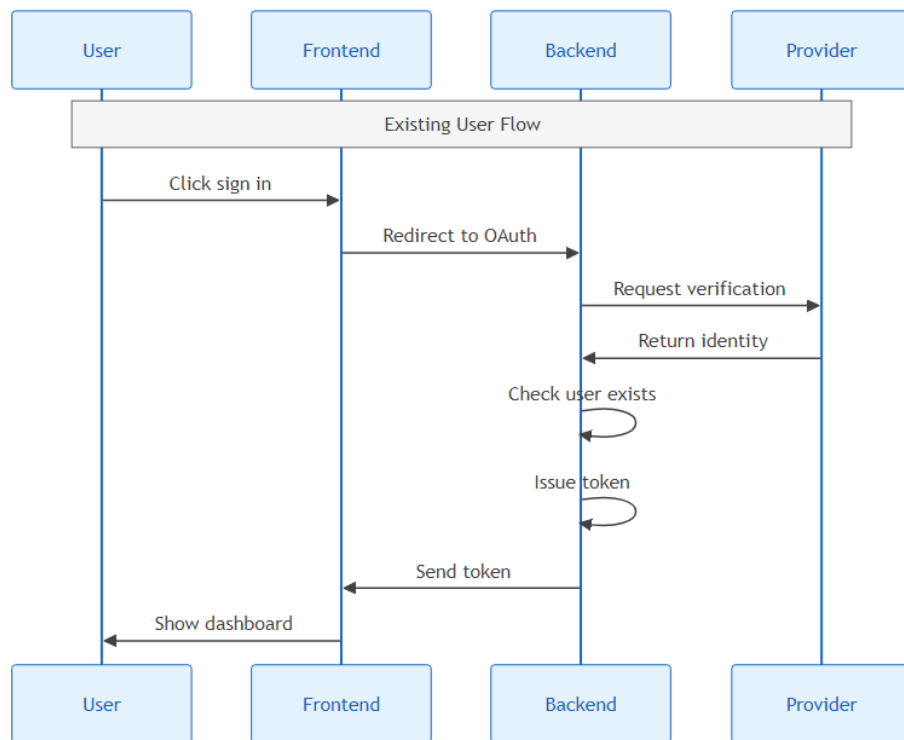


Figure 4.7: SSO Journey — Existing User

For existing users, authentication is immediate: after the identity provider confirms their identity, the backend looks up their account by email, invalidates any old sessions for security, issues a fresh token, and redirects the user to the dashboard.

4.1.1.4.2 New User Journey Figure 4.8 shows the flow for first-time SSO users who do not yet have an account.

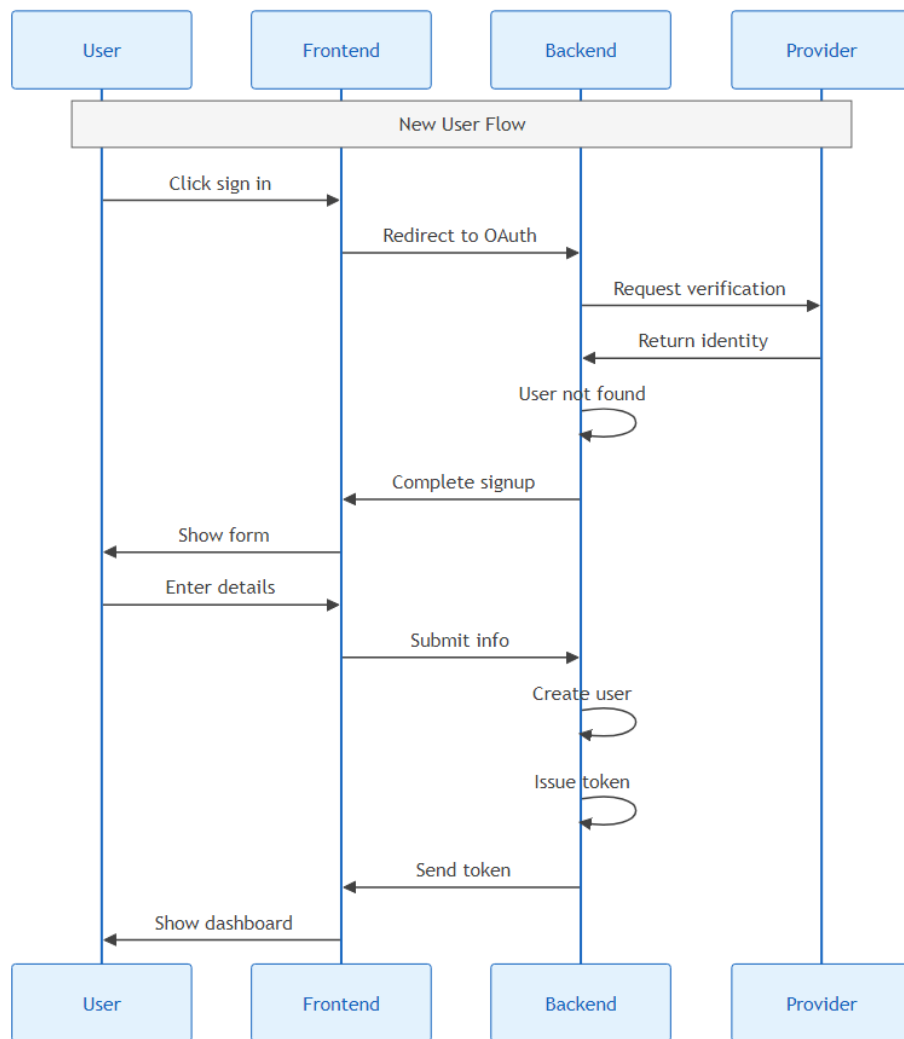


Figure 4.8: SSO Journey — New User (First-Time Sign-Up)

For new users, an additional step is required: since the system needs a password for traditional login fallback, the user receives a short-lived temporary token and is directed to a signup completion page. After setting a password and confirming their details, the system creates their account, issues a full authentication token, and redirects them to the dashboard.

4.1.1.5 Shared SSO Behaviour

Both Google and Apple SSO modules share a common authentication flow managed by the core Auth Module. Whether users authenticate via SSO or through traditional email/password login, they receive the same token-based session mechanism. This unified approach means that all users benefit from token blacklisting, single-session enforcement, and the same token refresh infrastructure regardless of how they initially authenticated.

4.1.2 Payment & Subscription Management

This subsection documents how the payment processing and subscription management system integrates Stripe, a leading payment processor, with a local database to track user

subscriptions and enforce plan limits. The implementation is organised into three core modules that work together to handle the complete lifecycle of user subscriptions.

4.1.2.1 Payment System Overview

Figure 4.9 illustrates the payment system architecture, showing how the three backend modules interact with Stripe and the data stores.

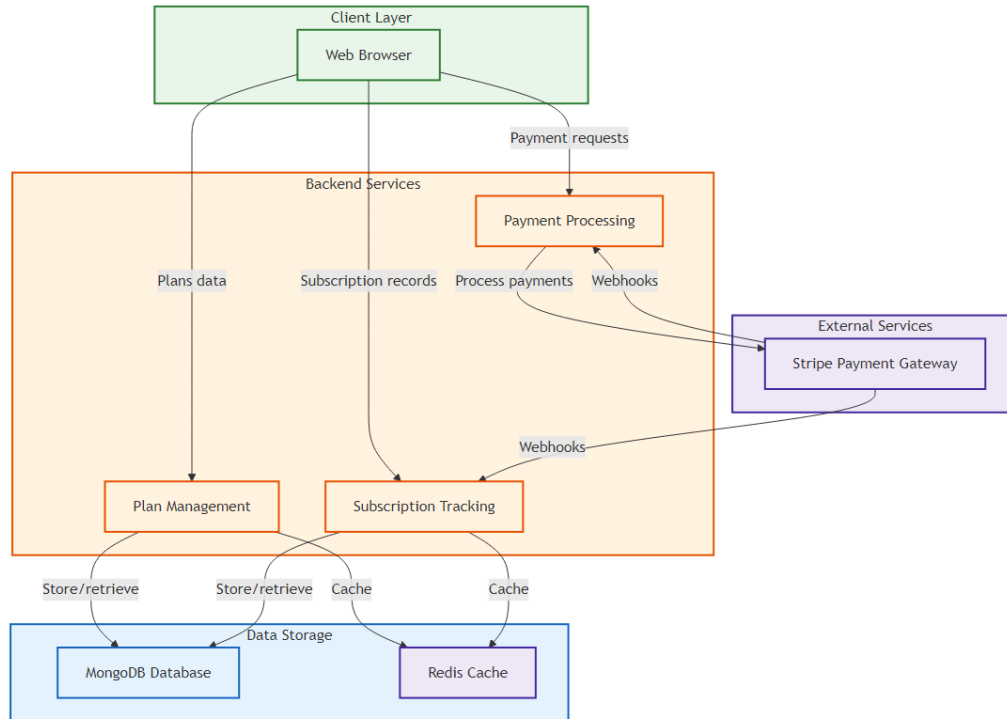


Figure 4.9: Payment System Architecture

The system consists of three modules that work in concert. The **Payment Processing** module acts as the bridge to Stripe’s infrastructure, handling checkout sessions, listening for payment events via webhooks, and retrieving pricing information. The **Subscription Tracking** module maintains a local record of each user’s subscription status, including their current plan, validity dates, and usage consumption against their quota. The **Plan Management** module stores the definitions of subscription tiers (Free, Basic, Pro) with their associated features and quota limits, using Redis caching for performance. Together, these modules ensure that payment information from Stripe is reflected in the application’s database, and that users’ access to features is governed by their active subscription state.

4.1.2.2 Checkout and Payment Flow

Figure 4.10 illustrates the step-by-step checkout process when a user subscribes to a plan.

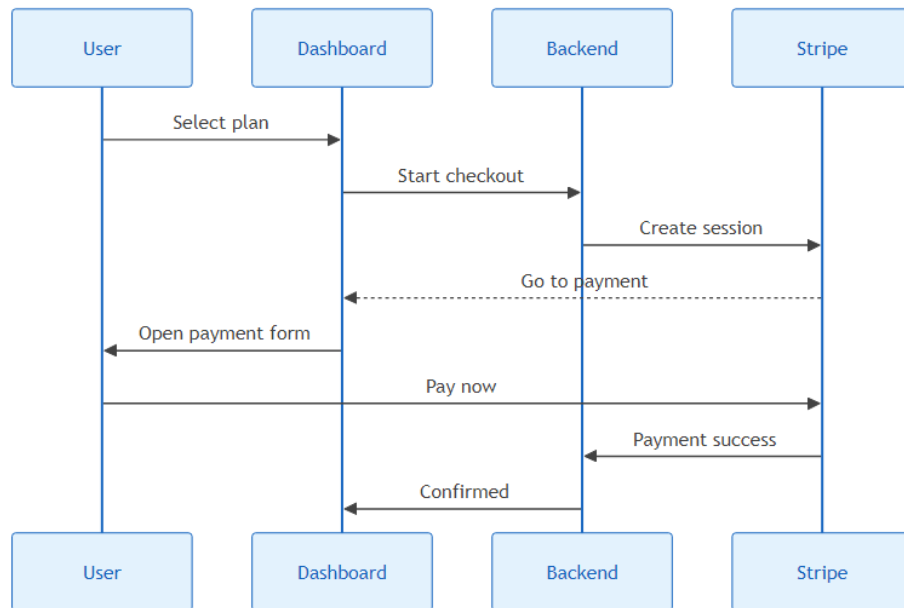


Figure 4.10: Stripe Checkout Flow

The flow begins when the user selects a subscription plan on the dashboard. The frontend requests a checkout session from the backend, which creates a secure payment session with Stripe. The user is then redirected to Stripe’s hosted payment page, where they enter their payment details in a secure environment managed entirely by Stripe. Upon successful payment, Stripe sends a confirmation notification (webhook) back to the backend, which activates the subscription in the local database. Finally, the user sees a confirmation message on the dashboard. This design keeps payment processing secure by delegating it entirely to Stripe while maintaining a synchronised local copy of subscription data for feature enforcement.

4.1.2.3 Subscription Lifecycle

A subscription passes through several states throughout its lifetime. Figure 4.11 shows these states and the transitions between them.

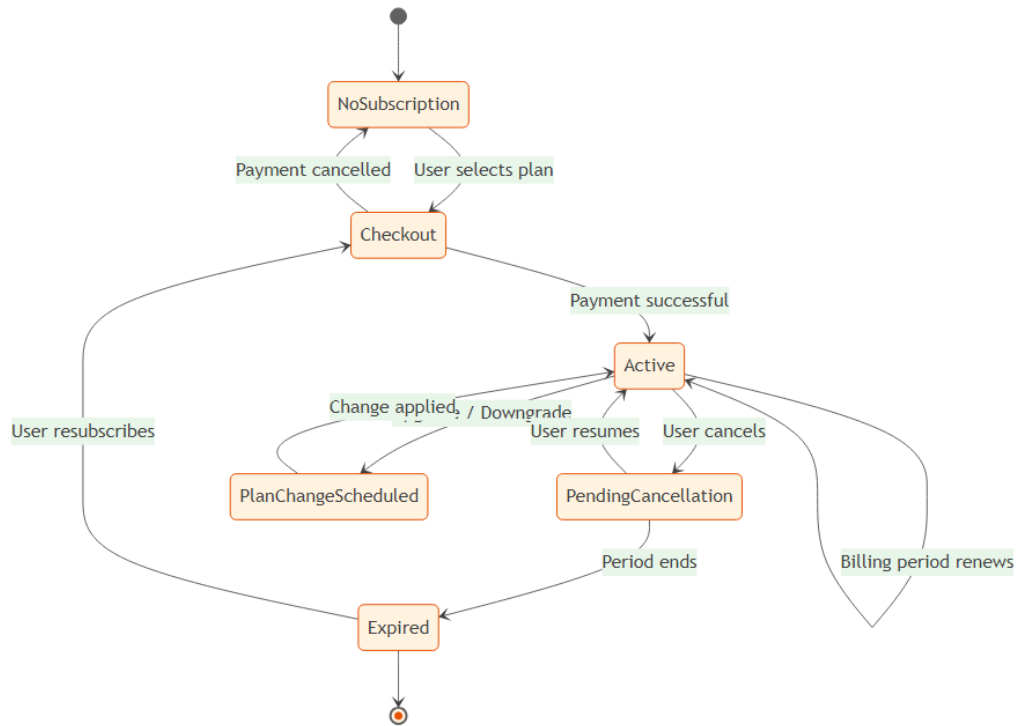


Figure 4.11: Subscription Lifecycle State Diagram

The lifecycle works as follows:

1. **New Subscription:** When payment completes, the system records the subscription as active, sets the access period (start and end dates), and initialises the user's usage quota based on the selected plan.
2. **Renewal:** At the end of each billing period, Stripe automatically charges the customer. The application detects this renewal event and extends the access period forward (e.g., by 30 days for monthly plans) while resetting usage counters to zero, giving the user a fresh allocation for the new period.
3. **Cancellation:** When a user requests cancellation, the subscription is marked as pending cancellation. The user retains full access until the current billing period naturally expires, providing a graceful experience.
4. **Expiration:** Once the billing period ends, the status changes to expired and the user loses access to premium features.
5. **Plan Change:** Users can schedule an upgrade or downgrade. Rather than applying changes immediately, the system schedules the new plan to take effect at the start of the next billing cycle, avoiding billing complications.

4.1.2.4 Plan Management and Versioning

Figure 4.12 illustrates how plan versions are managed when administrators update subscription tiers.

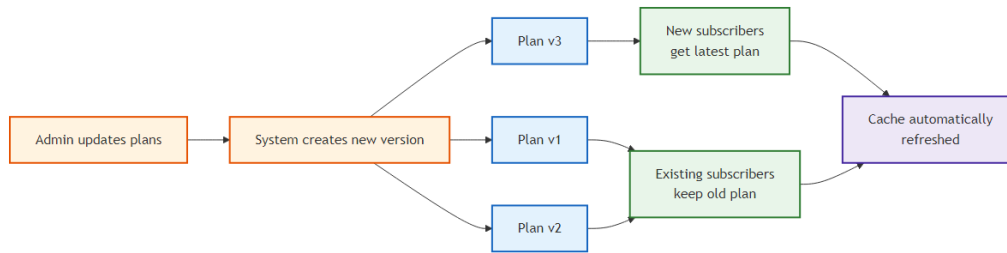


Figure 4.12: Plan Versioning and Subscriber Impact

When administrators need to modify plans—such as adjusting quota limits or updating feature descriptions—the system creates a new plan version rather than overwriting the old one. This approach preserves a historical record and ensures that existing subscribers continue to benefit from the terms of their originally purchased plan while new subscribers receive the updated terms. Plan data is heavily cached in Redis (refreshed automatically when new versions are created) to minimise database queries, since plan information is requested on every page load.

4.1.2.5 Webhook Synchronisation

Webhooks are critical for keeping the local subscription database synchronised with Stripe’s authoritative payment records. Figure 4.13 shows how the system processes different types of Stripe events.

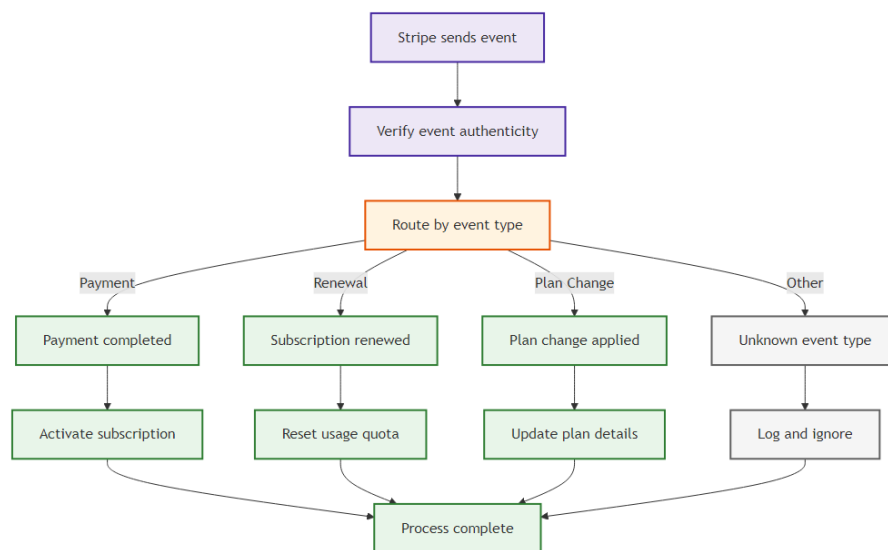


Figure 4.13: Stripe Webhook Processing Flow

Whenever a significant payment event occurs in Stripe, it sends a webhook notification to the application. The backend first verifies the message authenticity to ensure it genuinely comes from Stripe, then routes it based on event type. A **Payment Completed** event triggers subscription activation in the local database. A **Subscription Renewed** event resets usage quotas for the new billing period. A **Plan Change Applied** event updates the subscription to reflect the new plan and its associated quotas. Unrecognised events are logged and safely ignored. By using webhooks rather than polling Stripe repeatedly, the system achieves real-time synchronisation with minimal overhead.

4.1.2.6 Quota and Usage Tracking

The system enforces plan limits by tracking usage against quota. Each subscription record stores the total resource allowance per billing period (quota) and the amount already consumed (usage). Before allowing a resource-intensive operation such as transcribing audio, the system checks whether current usage exceeds the quota. This check occurs independently for each type of quota (e.g., batch processing minutes and live transcription minutes), allowing fine-grained control. When a subscription renews, usage counters reset to zero while quota limits remain fixed, giving users a fresh allocation each period.

4.1.2.7 Caching Strategy

To reduce latency and database load, the system caches frequently accessed data in Redis. Plan definitions are cached for one hour (since they change infrequently but are queried on every page load), and Stripe price information is cached for 24 hours to avoid repeated external API calls. Both caches use a protective technique: if a requested item does not exist, the system temporarily remembers that it is missing (for 60 seconds), preventing repeated lookups for non-existent items that could otherwise overwhelm the system under heavy load. The caching layer is transparent to the rest of the application and is automatically invalidated when administrators update plan information.

4.2 Frontend Implementation

The frontend is a Single Page Application (SPA) built with Vue.js 3 and Vuetify, providing an intuitive web-based interface for users and administrators. This section describes the frontend architecture, authentication experience, navigation system, and subscription management interface.

4.2.1 Overview

The Gateway Dashboard is a modern web application built with Vue.js 3, providing an intuitive management interface for the Speech Gateway API. The application supports user authentication, subscription management, and administrative controls through a responsive single-page application architecture.

4.2.1.1 Frontend Architecture

Figure 4.14 illustrates the layered architecture of the frontend application, showing the four main layers and how data flows between them.

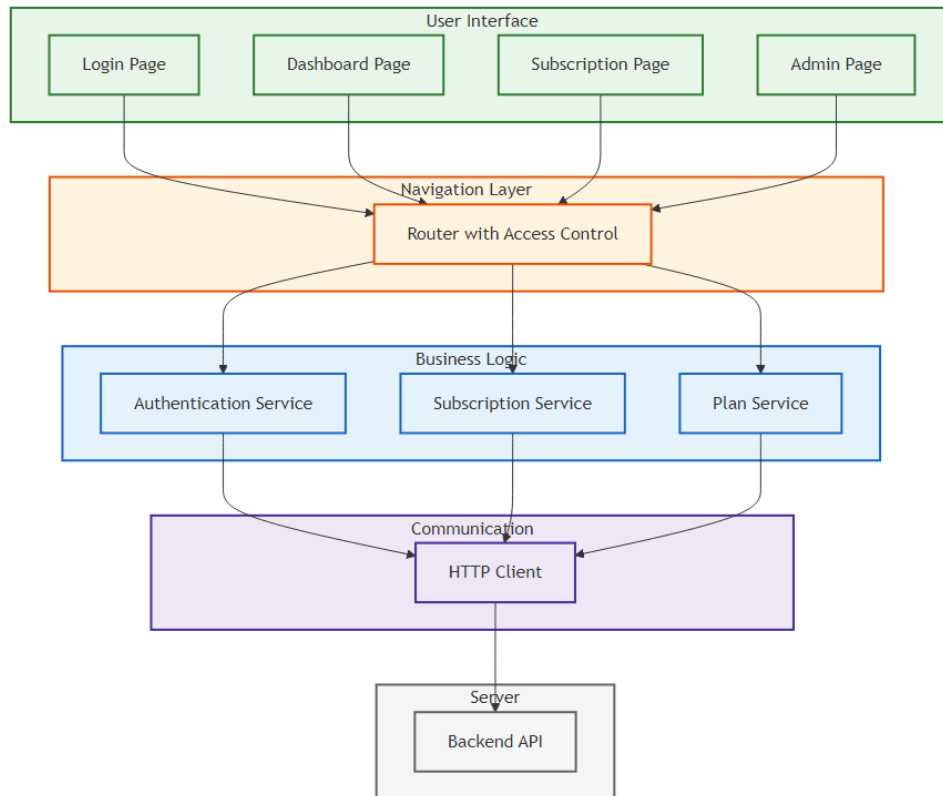


Figure 4.14: Frontend Application Architecture

The frontend follows a clean layered architecture with clear separation of concerns. At the top, the **User Interface** layer contains the pages that users interact with (Login, Dashboard, Subscription, and Admin). The **Navigation** layer manages which pages users can access based on their authentication status and role. The **Business Logic** layer handles authentication state, subscription data, and plan information. At the bottom, the **Communication** layer sends and receives data from the backend API, automatically attaching authentication credentials to every request.

This layered approach ensures that changes in one layer (for example, updating the UI design) do not affect the underlying business logic or communication mechanisms.

4.2.1.2 Technology Stack

Layer	Technology
Frontend Framework	Vue.js 3
UI Component Library	Vuetify (Material Design 3)
Build Tool	Vite
Routing	Vue Router 4

Table 4.1: Frontend Technology Stack

The application uses Vue.js 3 as the core framework for building reactive user interfaces. Vuetify provides a comprehensive set of Material Design components, ensuring a professional and consistent user experience. Vite serves as the build tool, enabling fast development and optimised production builds. Vue Router 4 manages navigation and enforces authentication constraints through navigation guards.

4.2.2 Authentication Frontend

The frontend authentication system provides a secure and user-friendly login experience with support for three authentication methods: email and password, Google OAuth, and Apple Sign-In. The system manages tokens securely, automatically refreshes expired credentials, and persists user sessions across browser sessions.

4.2.2.1 Three Authentication Methods

The application offers multiple login pathways to accommodate different user preferences:

- **Email/Password Authentication:** Users register and log in using credentials entered directly in the application. This traditional method is suitable for users who prefer direct credential management.
- **Google OAuth:** Users can authenticate using their existing Google accounts, avoiding the need for a separate password. The authentication flow redirects to Google's servers and returns control to the dashboard upon successful authentication.
- **Apple Sign-In:** Similar to Google OAuth, users authenticate through Apple's identity system, offering privacy-focused authentication with optional email masking.

All three methods result in the same token-based authentication system on the frontend, with no differences in subsequent operations or access control.

4.2.2.2 Token Management Strategy

The authentication system implements a token-based approach where the backend issues JSON Web Tokens (JWT) upon successful login. These tokens are stored in the browser's local storage for persistence across sessions.

4.2.2.2.1 Automatic Token Refresh To maintain session continuity without requiring frequent re-authentication, the system implements automatic token refresh. When a token expires, the application detects the expiration and requests a new token from the backend using a refresh mechanism. This process happens transparently without requiring user intervention. Tokens are verified periodically to detect expiration before they become invalid.

4.2.2.2.2 Session Persistence Along with the access token, the application stores metadata including subscription end dates and verification status in local storage. This allows the application to display relevant information immediately upon loading without waiting for API requests. When users close and reopen the browser, their authentication state is restored automatically if their token remains valid.

4.2.2.2.3 Error Handling If token refresh fails (for example, due to network issues or explicit token revocation), the application clears all stored credentials and redirects the user to the login page, forcing re-authentication.

4.2.2.3 OAuth Signup Flow

Users who authenticate through Google or Apple Sign-In but lack an existing account in the system must complete additional signup steps:

1. The backend identifies that the OAuth identity is new to the system
2. The user is redirected to a signup completion page
3. The user sets a password and provides their full name
4. Upon successful completion, the system stores the new account information and issues an access token
5. The user is automatically logged in and redirected to the main dashboard

This two-step process ensures that all users, regardless of authentication method, have a complete account with all necessary information stored in the system.

4.2.2.4 Authentication State Management

The application uses a reactive authentication composable that provides components with access to authentication state such as whether the user is logged in, their access token, subscription status, and verification status. This composable also exposes a logout function that revokes the user's token and clears all stored credentials.

The authentication system is separated from the API communication layer to avoid circular dependencies and ensure reliable token handling regardless of API state. This separation is maintained through a dedicated authentication service that uses low-level fetch mechanisms for authentication operations, while other API calls use a higher-level HTTP client with automatic token injection.

4.2.3 Routing and Navigation

Vue Router manages navigation throughout the application and enforces access control constraints. The routing system ensures that users can only access pages appropriate for their authentication status and role, with automatic redirection when access is denied.

4.2.3.1 Route Configuration

The application defines a set of routes corresponding to different pages and features. Each route specifies whether authentication is required and whether administrative privileges are necessary.

Path	Auth	Admin	Component
/	Yes	Yes	Dashboard.vue
/sign-in	No	No	SignIn.vue
/sign-up	No	No	SignUp.vue
/complete-signup	No	No	CompleteSignup.vue
/success	No	No	Success.vue
/cancel	No	No	Cancel.vue
/speeches	Yes	No	Speeches.vue
/speeches/:speechId	Yes	No	SpeechDetail.vue
/files	Yes	No	Files.vue
/files/:fileId	Yes	No	FileDetail.vue
/applications	Yes	No	Applications.vue
/applications/:appId	Yes	No	ApplicationDetail.vue
/subscription	Yes	No	Subscription.vue
/users	Yes	Yes	Users.vue
/users/:userId	Yes	Yes	UserDetail.vue
/request-log	Yes	Yes	RequestLog.vue
/settings	Yes	Yes	Settings.vue

Table 4.2: Route Definitions

Routes marked as requiring authentication cannot be accessed by unauthenticated users. Admin-only routes redirect non-administrative users to the regular dashboard.

4.2.3.2 Navigation Guards

Navigation guards are global checks that run before every page transition, ensuring users can only access pages they are authorised to view. Figure 4.15 illustrates this decision process.

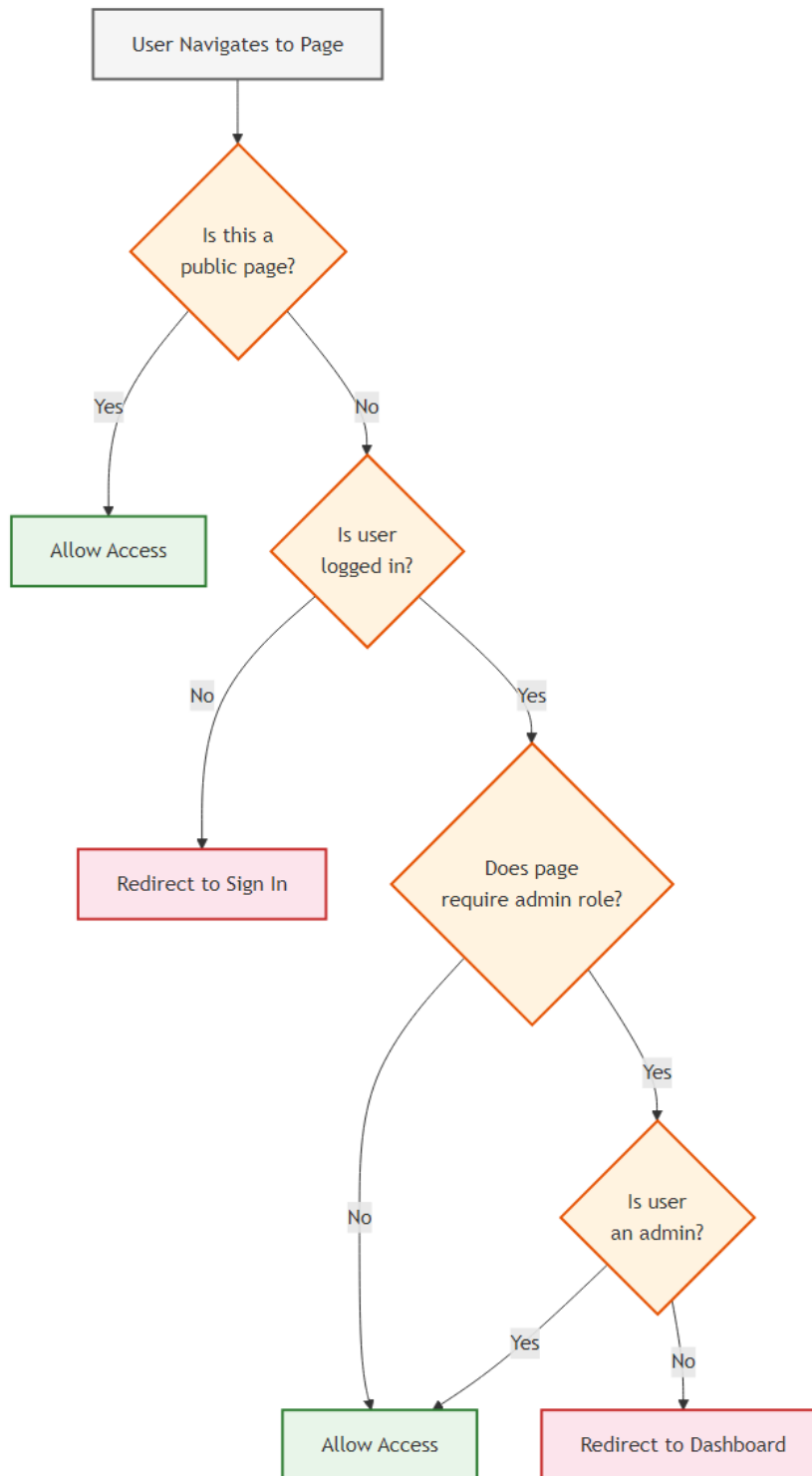
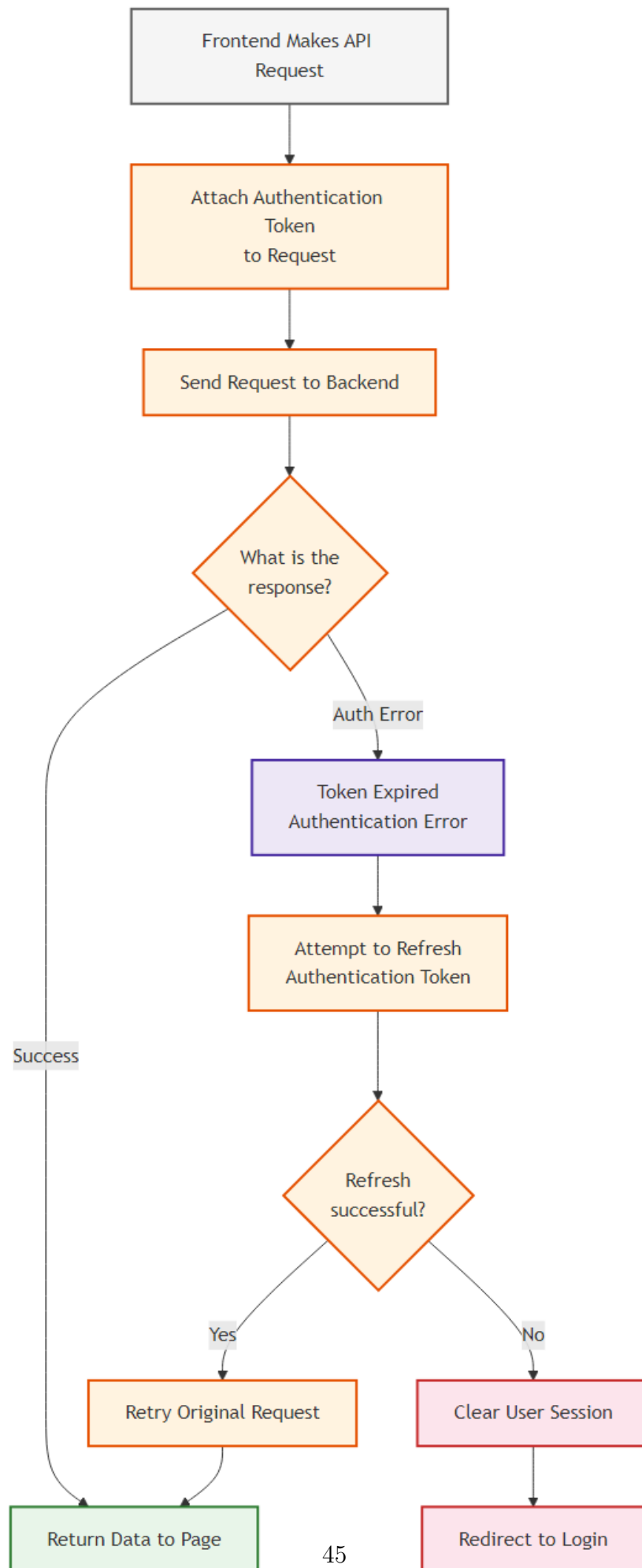


Figure 4.15: Navigation Guard Decision Flow

The guard evaluates three conditions in sequence. First, it checks whether the requested page is public (such as the sign-in or sign-up page)—if so, access is granted immediately. Second, for protected pages, it verifies that the user is logged in; if not, the user is redirected to the sign-in page. Third, for admin-only pages, it checks whether the logged-in user has administrator privileges; non-admin users are redirected to the regular dashboard. This layered approach ensures that every navigation action is validated before the page loads.

4.2.3.3 Automatic Token Management

The frontend automatically manages authentication tokens on all API requests, ensuring seamless operation without manual intervention. Figure 4.16 illustrates how this mechanism works.



When any page makes a request to the backend, the system automatically attaches the user's authentication token. If the backend accepts the request, the data is returned normally. However, if the token has expired, the system detects the authentication error and automatically attempts to obtain a fresh token from the backend. If the refresh succeeds, the original request is retried transparently—the user never notices the interruption. If the refresh fails (for example, because the user's session was revoked), the system clears all stored credentials and redirects the user to the sign-in page.

This approach provides a balance between security and user experience: tokens expire regularly for security purposes, but users are not unnecessarily interrupted unless re-authentication is truly required.

4.2.4 Subscription Management Interface

The subscription interface provides users with tools to select service plans, complete payments through Stripe, and monitor their current usage quotas. Administrators can manage available plans through a dedicated configuration interface. The entire subscription system is built with Vue 3 components providing a responsive and interactive experience.

4.2.4.1 User Subscription View

For users with active subscriptions, the dashboard displays comprehensive subscription information:

- Current plan name and subscription billing period (start and end dates)
- Quota allocation for both batch and real-time processing (measured in seconds or unlimited)
- Visual progress indicators showing current usage relative to allocated quotas
- Color-coded usage indicators: green for low usage, orange for moderate usage (75%), and red for high usage (90%+)
- Controls for canceling or resuming subscription at any time
- Scheduled plan changes with information about the next billing cycle

Usage percentages are calculated to provide clear feedback on consumption. Unlimited quotas are represented as zero usage to indicate no constraint. Plans with zero quota are displayed at 100% to indicate the service is not available under that plan.

4.2.4.2 Plan Card Component

The plan card is a reusable interface component that displays individual subscription plan options. Each plan card shows:

- Plan name and pricing retrieved from Stripe's pricing system
- Billing interval (monthly, annual, or one-time)
- Processing duration limits for both batch and live modes
- Feature list highlighting capabilities included in the plan

- Interactive button indicating action available: "Subscribe Now" for unsubscribed users, "Current Plan" for the active subscription, or "Schedule Change" to switch plans
- Badges indicating current or scheduled subscription status

Plan pricing is fetched dynamically from Stripe's API, ensuring the interface always displays current pricing without requiring manual updates. This integration allows changes to pricing in Stripe to be reflected immediately in the user interface.

4.2.4.3 Checkout Process

When users select a plan to purchase, the application initiates a seamless checkout process:

1. User clicks the subscribe button on the desired plan card
2. Application requests a checkout session from the backend, passing the Stripe price identifier
3. User is redirected to Stripe's hosted checkout page
4. User completes payment through Stripe's secure payment interface
5. After successful payment, Stripe redirects the user back to the dashboard
6. Application displays a success page and the subscription is activated

If users cancel during checkout, they are returned to the dashboard on a cancellation page, allowing them to retry or select a different plan. The checkout process is entirely handled by Stripe, ensuring payment security.

4.2.4.4 Subscription Management

After purchasing a subscription, users can manage their subscription through the subscription page:

4.2.4.4.1 Plan Changes Users with active subscriptions can switch to different plans. The system allows scheduling plan changes at the next billing cycle rather than forcing immediate upgrades or downgrades. This prevents unexpected billing changes and allows users to plan their changes in advance.

4.2.4.4.2 Cancellation Users can cancel their subscriptions at any time. Cancellation is scheduled for the end of the current billing period, allowing the user to retain access for the remainder of their paid cycle. The system displays clear information about when the subscription will end.

4.2.4.4.3 Resumption Users who have canceled their subscription can resume it before the cancellation takes effect. This allows users to change their mind without having to re-enter payment information.

4.2.4.5 Admin Plan Configuration

Administrators access a dedicated plan management interface for configuring available subscription plans. This interface provides:

- A JSON-based editor for defining plan properties including names, features, durations, and Stripe price identifiers
- Live validation to ensure pricing identifiers correspond to valid Stripe prices before publication
- Diff comparison showing what has changed from the current published version
- Version history tracking with ability to revert to previous configurations
- Real-time syntax highlighting and validation feedback

The admin editor validates each plan's Stripe price identifier against Stripe's API before allowing the configuration to be published, preventing invalid or broken pricing configurations from being saved.

Chapter 5

Conclusion and Future Work

This chapter synthesizes the project achievements, articulates the conclusions drawn from the implementation, and outlines promising directions for future development.

5.1 Project Summary

The Gateway Dashboard represents a comprehensive solution designed to transform how users interact with the Speech Gateway API. By consolidating authentication, payment processing, and service management into a unified web-based platform, the project bridges the gap between a powerful backend API and its practical, commercial deployment.

The implementation consists of three interrelated systems working in concert. The authentication layer provides multiple pathways for users to securely gain access to the platform, eliminating friction from the onboarding process while maintaining industry-standard security practices. The payment infrastructure automates billing and subscription management, enabling the speech services platform to operate as a sustainable commercial offering. Finally, the frontend dashboard presents a professional, user-friendly interface that makes advanced API features accessible to users without requiring technical expertise or direct API knowledge.

The architecture reflects careful consideration of both immediate business needs and long-term scalability. By separating the user-facing frontend from the backend services, the system can evolve independently—allowing for iterative improvements to the user experience without disrupting the underlying business logic.

5.2 Conclusions

This project successfully delivers a production-ready platform that fundamentally advances the viability of the Speech Gateway API as a commercial service offering. Several key achievements merit emphasis:

First, the multi-provider authentication system—supporting email, Google, and Apple credentials—significantly lowers barriers to user adoption by accommodating diverse authentication preferences. This flexibility is increasingly expected in modern applications and directly impacts user conversion rates. The security measures embedded throughout this system, including token management and session protection, ensure that user trust is not compromised in pursuit of convenience.

Second, the integration with Stripe establishes automated billing and subscription management capabilities that were previously absent. This addresses a fundamental busi-

ness requirement: the ability to reliably monetize the platform. The subscription system enforces access quotas automatically, creating clear service tiers that users can understand and select based on their needs. This infrastructure transforms the Speech Gateway from a technical capability into a viable commercial product.

Third, the dashboard itself represents a democratization of the API. Rather than requiring developers to understand REST endpoints and authentication tokens, users can now manage their subscriptions, monitor usage, and access documentation through an intuitive web interface. This dramatically expands the potential market beyond technically sophisticated users to include organizations that depend on non-technical stakeholders for service management.

The modular design of the implementation—separating authentication, payment, and user interface concerns—ensures that the system remains maintainable as business requirements evolve. The decision to adopt industry-standard technologies and frameworks ensures that future developers can build upon this foundation without requiring specialized knowledge.

Most significantly, this project establishes that the Speech Gateway can operate as a sustainable, user-accessible commercial service. The combination of secure authentication, reliable billing, and professional user interfaces creates the foundation necessary for growth and market adoption.

5.3 Future Work

While the current implementation establishes a strong commercial foundation, several strategic areas present opportunities for expansion and refinement.

In terms of user experience, the most impactful enhancement would be expanded language support and advanced usage analytics. Organizations managing the platform on behalf of their teams would benefit from team accounts that consolidate billing and user management under organizational structures rather than individual users. A mobile application would extend the platform’s accessibility beyond the desktop context.

From a business perspective, flexible billing models merit consideration. Beyond the current subscription approach, implementing usage-based billing would accommodate customers with variable demand patterns, potentially expanding the addressable market. Support for promotional campaigns and invoicing workflows would enhance enterprise sales capabilities.

Infrastructure improvements would primarily focus on deployment and operational reliability. Containerization would standardize deployment across environments, while automated testing and deployment pipelines would reduce operational overhead. Enhanced monitoring capabilities would provide visibility into system performance and user behavior in production.

5.4 Final Remarks

The Gateway Dashboard project demonstrates the practical application of modern web technologies to address real business challenges. By implementing secure authentication, reliable billing, and professional user interfaces, the project establishes that the Speech Gateway API can operate as a sustainable commercial service.

The success of this project lies not merely in the technical sophistication of the implementation, but in its focus on commercial viability and user accessibility. The system

enables organizations to adopt speech processing capabilities without requiring extensive technical integration efforts. This shift from technical capability to accessible service represents a meaningful advance in the platform’s market potential.

The architectural decisions and design patterns established in this project provide a template for developing similar service platforms. More broadly, the project demonstrates how thoughtful platform design—combining security, reliability, and user experience—can transform a powerful API into a commercially viable product offering.

Bibliography

- [1] Apple Inc. *Apple Platform Security*. <https://support.apple.com/guide/security>. 2023.
- [2] Apple Inc. *Sign in with Apple REST API*. <https://developer.apple.com/documentation/authenticationservices>. 2023.
- [3] Apple Inc. *Sign in with Apple: Technical Overview*. <https://developer.apple.com/sign-in-with-apple>. 2019.
- [4] Kyle Banker. *MongoDB in action*. Manning Publications, 2011.
- [5] Adam Barth, Collin Jackson, and John C Mitchell. “Securing frame communication in browsers”. In: *17th USENIX Security Symposium*. 2008, pp. 17–30.
- [6] Baymard Institute. *E-Commerce Checkout Usability: An Original Research Study*. <https://baymard.com/checkout-usability>. 2020.
- [7] Gavin Bierman, Martín Abadi, and Mads Torgersen. “Understanding TypeScript”. In: *European Conference on Object-Oriented Programming*. 2014, pp. 257–281.
- [8] Josiah L Carlson. *Redis in Action*. Manning Publications, 2013.
- [9] Kristina Chodorow. *MongoDB: the definitive guide*. O’Reilly Media, 2013.
- [10] Chee Yong Chong and Sai Peck Lee. “Component-based software engineering: Technologies, development frameworks, and quality assurance schemes”. In: *Information and Software Technology* 107 (2019), pp. 70–86.
- [11] Roy Thomas Fielding. “Architectural styles and the design of network-based software architectures”. PhD thesis. University of California, Irvine, 2000.
- [12] Martin Fowler. *Patterns of enterprise application architecture*. Addison-Wesley, 2003.
- [13] Zheng Gao, Christian Bird, and Earl T Barr. “Type systems for JavaScript: A survey”. In: *Software Engineering at Google* (2017), pp. 1–27.
- [14] Gartner Inc. *Market Guide for Cloud Workspace Productivity Suites*. 2023.
- [15] Annabelle Gawer and Michael A Cusumano. *Platform leadership: How Intel, Microsoft, and Cisco drive industry innovation*. Harvard Business School Press, 2014.
- [16] GitHub Inc. *GitHub Annual Report 2023*. <https://github.com/about>. 2023.
- [17] Google Inc. *Google Account Security Features*. <https://safety.google/authentication>. 2023.
- [18] Google Inc. *Google Services: Active User Statistics 2023*. <https://about.google>. 2023.
- [19] Google Inc. *Material Design Guidelines*. <https://material.io/design>. 2014.
- [20] Google Inc. *Using OAuth 2.0 to Access Google APIs*. <https://developers.google.com/identity/protocols/oauth2>. 2023.

- [21] Wenjuan Guo et al. “Freemium as an optimal strategy for market dominant firms”. In: *Marketing Science* 38.1 (2019), pp. 150–169.
- [22] Rahul Gupta. *Stripe integration: A comprehensive guide for developers*. 2020.
- [23] Dick Hardt. *The OAuth 2.0 authorization framework*. Tech. rep. RFC 6749, 2012.
- [24] Amir Herzberg and Ahmad Jbara. “Payment system integration: A comparative study of Stripe and PayPal”. In: *Journal of Electronic Commerce Research* 20.3 (2019), pp. 145–162.
- [25] Jim Isaak and Mina J Hanna. “User data privacy: Facebook, Cambridge Analytica, and privacy protection”. In: *Computer* 51.8 (2018), pp. 56–59.
- [26] Michael Jones, John Bradley, and Nat Sakimura. *JSON web token (JWT)*. Tech. rep. RFC 7519, 2015.
- [27] Neal Leavitt. “Will NoSQL databases live up to their promise?” In: *Computer* 43.2 (2010), pp. 12–14.
- [28] Cheng Li et al. “Building reliable distributed systems using message queues”. In: *Proceedings of the 25th Symposium on Operating Systems Principles*. 2015, pp. 239–254.
- [29] Torsten Lodderstedt, Mark McGloin, and Phil Hunt. “OAuth 2.0 threat model and security considerations”. In: *RFC 6819* (2013).
- [30] Callum Macrae. *Vue.js: Up and Running*. O’Reilly Media, 2018.
- [31] Mark Masse. *REST API design rulebook*. O’Reilly Media, 2011.
- [32] Microsoft Corporation. *Microsoft Identity Platform Documentation*. <https://docs.microsoft.com/azure/active-directory>. 2023.
- [33] Michael S Mikowski and Josh C Powell. *Single Page Web Applications: JavaScript end-to-end*. Manning Publications, 2013.
- [34] Sam Newman. *Building microservices: designing fine-grained systems*. O’Reilly Media, 2015.
- [35] Addy Osmani. *A comparison of JavaScript frameworks*. 2017.
- [36] Addy Osmani. *Web performance optimization*. 2017.
- [37] OWASP Foundation. *OWASP Top 10 - 2021: The Ten Most Critical Web Application Security Risks*. Tech. rep. OWASP, 2021.
- [38] Suhas Pai et al. “SoK: Systematization of knowledge on web single sign-on systems”. In: *2011 IEEE Symposium on Security and Privacy*. IEEE. 2011, pp. 232–246.
- [39] Cesare Pautasso, Olaf Zimmermann, and Frank Leymann. “RESTful web services vs. ”big” web services: making the right architectural decision”. In: *Proceedings of the 17th international conference on World Wide Web* (2008), pp. 805–814.
- [40] PayPal Holdings Inc. *PayPal Q4 2023 Earnings Report*. <https://investor.ppyl.com>. 2023.
- [41] Ken Peffers, Charles E Gengler, and Tuure Tuunanen. “Electronic payment systems: an analysis and comparison of types”. In: *IEEE Transactions on Engineering Management* 50.3 (2003), pp. 296–304.
- [42] Tomas Petricek, Gustavo Guerra, and Don Syme. “Types from data: Making structured data first-class citizens in F#”. In: *ACM SIGPLAN Notices* 51.6 (2016), pp. 477–490.

- [43] Chris Richardson. *Microservices patterns: with examples in Java*. Manning Publications, 2018.
- [44] Carlos Rodriguez et al. “RESTful web services: A study and comparison of tools”. In: *Service Oriented Computing and Applications* 10 (2016), pp. 435–450.
- [45] Nat Sakimura, John Bradley, and Naveen Agarwal. *Proof Key for Code Exchange by OAuth Public Clients*. Tech. rep. RFC 7636, 2015.
- [46] Günther Schuh, Michael Riesener, and Christof Mattern. “Subscription business models: Growth and profitability”. In: *Production Engineering* 7 (2013), pp. 385–392.
- [47] Prabath Siriwardena. *Advanced API security: OAuth 2.0 and beyond*. Apress, 2020.
- [48] S Mahbub Sohan. “A pattern language for webhooks”. In: *PLoP’17: 24th Conference on Pattern Languages of Programs* (2017).
- [49] StatCounter Global Stats. *Mobile Operating System Market Share Worldwide*. <https://gs.statcounter.com>. 2023.
- [50] Statista Research Department. *Most popular online accounts worldwide 2023*. <https://www.statista.com>. 2023.
- [51] Stripe Inc. *Stripe API Documentation*. <https://stripe.com/docs/api>. 2023.
- [52] Stripe Inc. *Stripe Billing Documentation*. <https://stripe.com/docs/billing>. 2023.
- [53] Stripe Inc. *Stripe Security and Compliance*. <https://stripe.com/docs/security>. 2023.
- [54] Yi Sun et al. “Understanding user adoption of single sign-on systems”. In: *Computers & Security* 83 (2019), pp. 143–154.
- [55] Yuhui Sun, Yan Zhang, and Wei Peng. “Security and privacy for web databases and services”. In: *Web Database Systems: Technologies and Applications* (2010), pp. 15–37.

Appendix A

This section will be completed in the final report, once everything has been finalized.